



**SEVA MANDAL EDUCATION SOCIETY'S
SMT.KAMALABEN GAMBHIRCHAND SHAH
DEPARTMENT OF COMPUTER APPLICATIONS
UNDER
DR.BMN COLLEGE OF HOME SCIENCE(Autonomous)
NAAC RE-ACCREDITED 'A+' GRADE WITH CGPA 3.69/4
UGC STATUS-COLLEGE WITH POTENTIAL FOR EXCELLENCE 338,R.A
KIDWAI ROAD, MATUNGA,MUMBAI-19
2023-2024
PROJECT ON
"FRUIT INVENTORY MANAGEMENT SYSTEM"
FOR
"DNJ MERCHANT AND COMMISSION AGENT"
UNDER GUIDANCE OF
PROF.NITIN PAWAR
submitted in partial fulfilment of the requirements for the
Degree Bachelor of Computer Application (BCA)**

Submission Date: 21st April 2024

**Submitted By:
Tanishq Verma(135)
Vaishnavi Ghadigaonkar(138)
Sanika Saravade(140)
Sakshi Sonawane(141)**

M/s. DASHRATH NAMDEO JADHAV

मे. दशरथ नामदेव जाधव

DNJ
G-WING

फ्रूट मर्चेंट आणि कमिशन एजेंट

जी - विंग, ए. पी. एम. सी. फ्रूट मार्केट, सेक्टर - १९, तुर्भे, वाशी, नवी मुंबई - ७०५.

Fruit Merchant and Commission Agent

G-Wing, A.P.M.C. Fruit Market, Sector-19, Turbhe, Vashi, Navi Mumbai-705.



CONFIRMATION LETTER

Mr. Nitin Pawar,
Subject Incharge
Dr. B. M. N. College of Home Science,
Matunga, Mumbai

Subject:-Project for students of BCA (Bachelor of Computer Applications)

Dear Sir,

This is to certify that the following students from your college are developing a Desktop Application for our shop located in Vashi, Navi Mumbai which is **Fruit Inventory Management system** for our company **"DNJ Fruit Merchant and Commission Agent"**

1. Tanishq Verma
2. Vaishnavi Ghadigaonkar
3. Sanika Saravade
4. Sakshi Sonawane

We have allowed them to develop an application and provide them with necessary details required for an application. They have taken good initiative from their end for the same. I really appreciate all of them for their excellent and enthusiastic work, and my best wishes to them.

Thank you.

कृषि उत्पन्न बाजार
समिती वाशी (नवी मुंबई)
प्रो : दशरथ नामदेव जाधव (DNJ)
(फ्रूट मार्केट अॅण्ड कमिशन एजेंट)

Owner of DNJ

DNJ Fruit Merchant and Commission Agent

ACKNOWLEDGEMENT

We would like to express our deep gratitude to range of people who provided us valuable support in this project. Their enthusiastic guidance and encouragement have helped us in making this project a great success.

Our sincere sentiments of appreciation and gratitude to our principle. prof. Dr. Mala Pandurang for the approval of project.

We want to thank Mr. Shahjahan Khan, head of the Department of Smt. Kamalaben Gambhirchand Shah Department of Computer Applications.

Also, We would like to express our heartiest gratitude to our teachers Mr. Nitin Pawar and Ms.Manjot Kaur for their valuable suggestions and encouragement and also undivided and continuous support during tenure of this project.

We would like to thank Mr. Bhaskar Saravade for the reference and for taking the time for his busy schedule to provide us with necessary details about their construction company business.

Finally a vote of thanks to the Department of Bachelor of Computer Applications, the staff, both teaching and non-teaching for their co-operation extended to us



SEVA MANDAL EDUCATION SOCIETY'S
SMT.KAMALABEN GAMBHIRCHAND SHAH
DEPARTMENT OF COMPUTER APPLICATIONS
UNDER

DR.BMN COLLEGE OF HOME SCIENCE(Autonomous)
NAAC RE-ACCREDITED 'A+' GRADE WITH CGPA 3.69/4
UGC STATUS-COLLEGE WITH POTENTIAL FOR EXCELLENCE 338,R.A KIDWAI ROAD,
MATUNGA,MUMBAI-19

CERTIFICATE

This to certify that **Ms.Tanishq Verma, Ms.Vaishnavi Ghadigaonkar, Ms.Sanika Saravade, Ms.Sakshi Sonawane** have completed the project titled **"FRUIT INVENTORY MANAGEMENT SYSTEM"** satisfactorily, and submitted the project report as per the guidelines of the S.N.D.T Women's University, Mumbai.

External Examiner:
Date: 21st April 2024

Program Coordinator: Mr. Shahjahan Khan
Date:

Principal Prof. Dr. Mala Pandurang
Date:

College Seal:

Subject in-charge: Mr. Nitin Pawar
Date:

INDEX

SR.NO	TITLE	PG.NO
1	INTRODUCTION	6
2	ORGANIZATION PROFILE	7
3	CURRENT SYSTEM AND DRAWBACKS	8
4	PROPOSED SYSTEM	9
5	DATA FLOW DIAGRAM	10
6	DATABASE DESIGN	14
7	INPUT,OUTPUT SCREEN ,TEST CASES AND CODING	23
9	SYSTEM REQUIREMENTS	125
10	CONCLUSION	126
11	SCOPE FOR THE FUTURE ENHANCEMENT	127
12	BIBLIOGRAPHY	128

INTRODUCTION

PROJECT TITLE:

Fruit Inventory Management System

OBJECTIVE:

The Fruit Inventory Management System aims to enhance the efficiency and effectiveness of the dealer's business operations by computerizing customer and supplier order details. Specifically designed for the management of fruit supplies, the system processes transactions and stores the results, facilitating the generation of data reports.

DEVELOPMENT TOOL AND TECHNOLOGY USED:

The Fruit Inventory Management System is a desktop-based Windows application developed using the C# programming language within the Microsoft .NET framework. The system utilizes a Microsoft SQL Database for efficient data management.

LANGUAGE: C#.Net

DATABASE: SQL

SOFTWARE'S: Visual studio 2022, Microsoft SQL Server Management Studio 2018.

ORGANIZATION PROFILE

Name of the Company: DNJ Fruit Merchant and Commission Agent

Owner: Dashrath Namdev Jadhav

Date of Establishment: 1980

Address: Vashi, Navi Mumbai

DNJ Fruit Merchant and Commission Agent is a business located at A.P.M.C Fruit Market, Vashi, Navi Mumbai. DNJ Fruit Merchant and Commission Agent operates as a middle agent, purchasing fruits from suppliers and selling them to customers. The business was established in 1980 at APMC Fruit Market, Vashi, Navi Mumbai. Since its inception, DNJ Fruit Merchant and Commission Agent has been providing high-quality goods and services at wholesale prices to its customers.

A few reputed customers of the business include:

1. Vikas, Panvel
2. Vishal Fruit Shop, Dombivli
3. Yogesh Modhve, Vashi

CURRENT SYSTEM AND DRAWBACKS

CURRENT SYSTEM:

The existing system relies on manual procedures, where all records, including customer details, supplier details, order details, and product records, are stored manually in file registers. The maintenance of records over an extended period becomes challenging. The current process for the agent involves lengthy and tedious manual record-keeping. Retrieving specific records when required poses difficulties, creating obstacles for business partners who need access to the records.

Locating

particular records and tracking sales manually is a time-consuming task.

DRAWBACKS:

- The manual maintenance of details in registers is labour-intensive and time-consuming.
- Entry and record-checking processes are time-consuming when done manually.
- Modifying the records is not very easy since for a minute change all the records would be required to be re-entered
- It also takes much time for doing entry and checking the records.
- Storing information in registers requires significant physical storage space.
- The need for new registers each year adds to the administrative burden.

PROPOSED SYSTEM

Our project aims to streamline and modernize the current manual processes of DNJ Fruit Merchant and Commission Agent through the implementation of a Fruit Inventory Management System. This system will leverage technology to automate various tasks and improve overall efficiency. Our proposed system is simple and without complexity at the same time SQL server provides the features such as primary key, foreign key. Visual Basic is used as a Front end which will provide a strong graphic user interface along with a user-friendly environment.

Advantages of the proposed system include:

Centralized Database:

Introduce a centralized database to store customer details, supplier details, order details, and product records electronically.

User-Friendly Interface:

Develop a user-friendly interface for easy navigation and efficient data entry.

Automated Record-Keeping:

Implement automated processes for record-keeping, reducing the labour-intensive nature of manual maintenance

Search and Retrieval Functionality:

Incorporate a robust search and retrieval functionality to quickly locate specific records, facilitating easy access for business partners.

Reports Generation:

Implement a reporting system that allows for the generation of comprehensive reports, including sales reports, customer trends, and inventory status.

Security Measures:

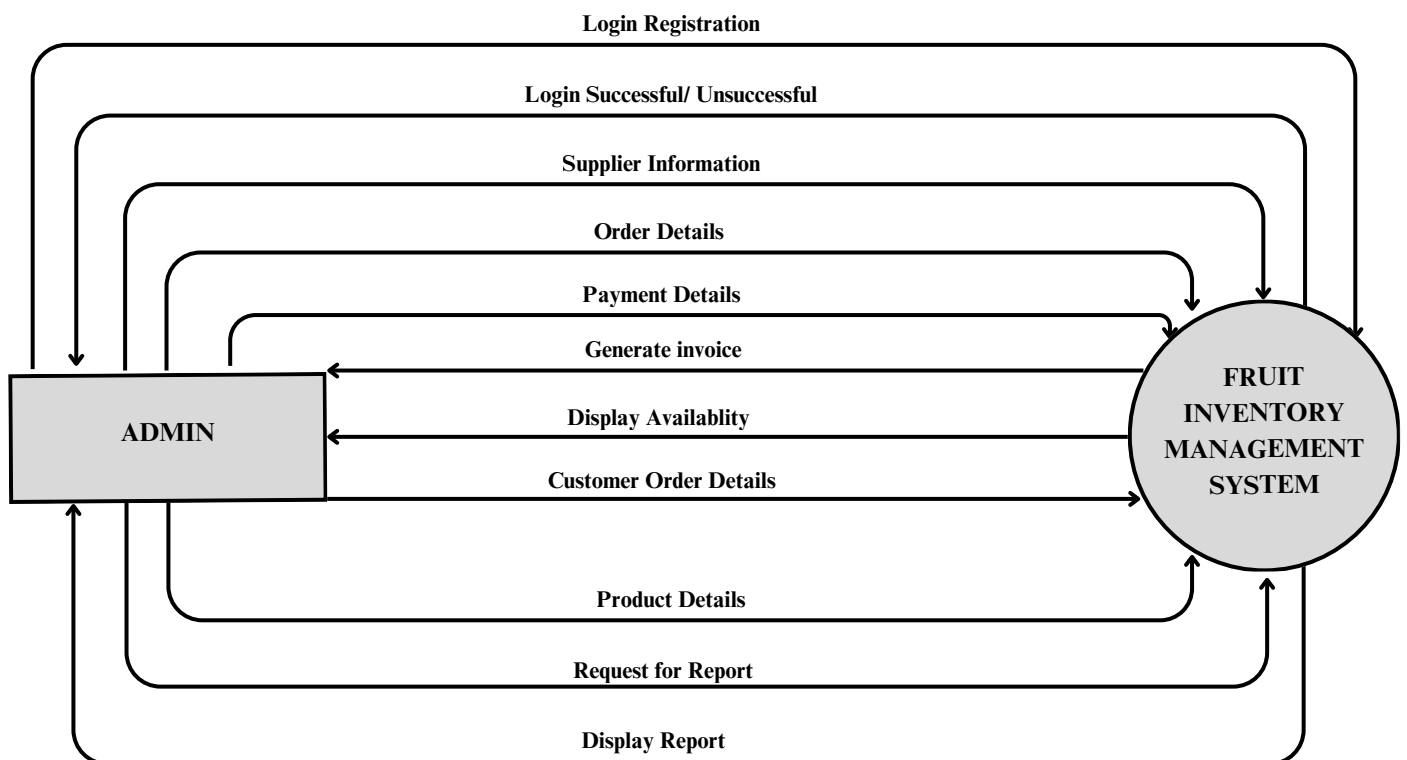
implement security measures to safeguard sensitive business data, ensuring confidentiality and integrity.

Printing:

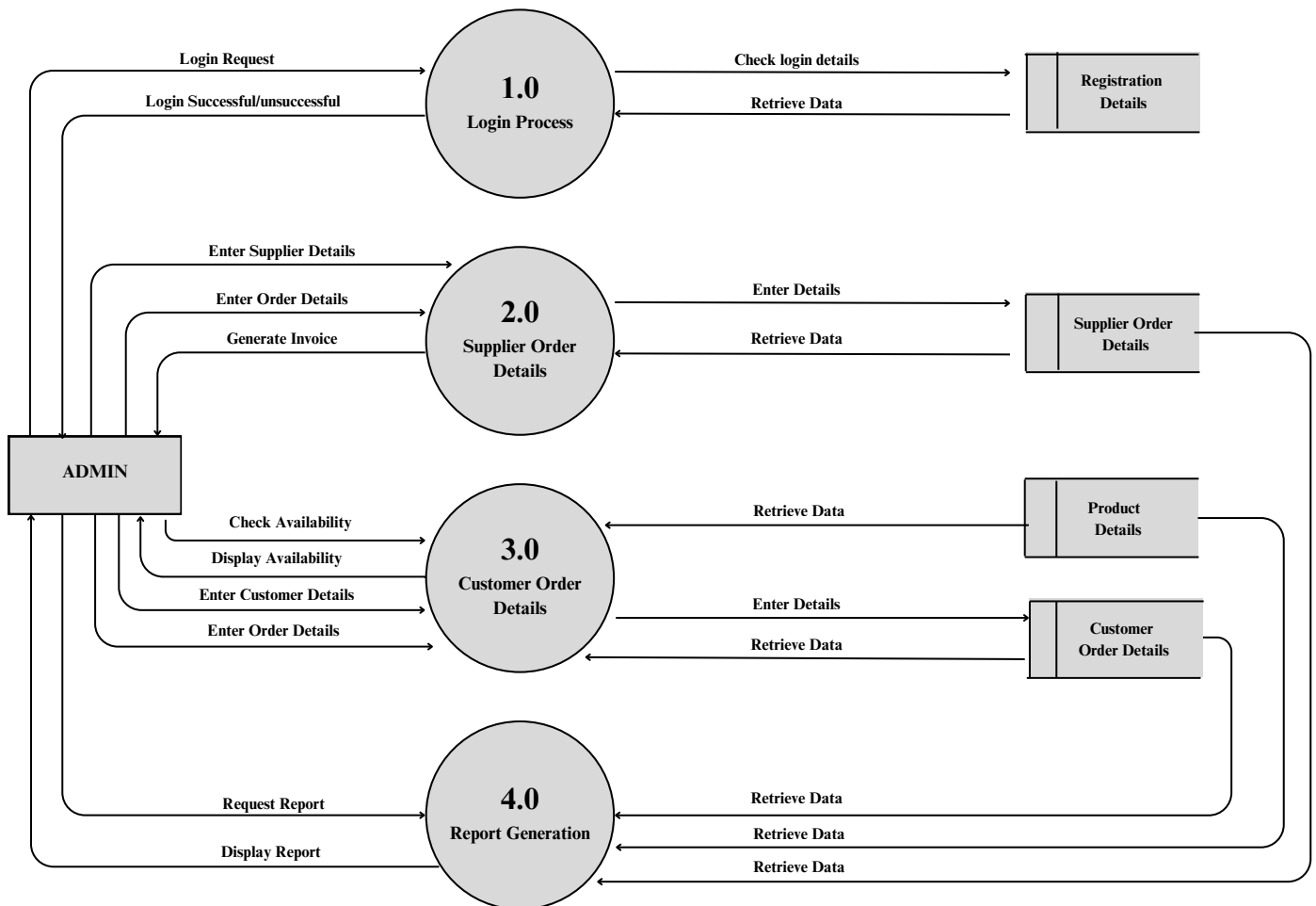
We can avail printer facilities., i.e., users can easily print weekly and monthly reports.

DATA FLOW DIAGRAM

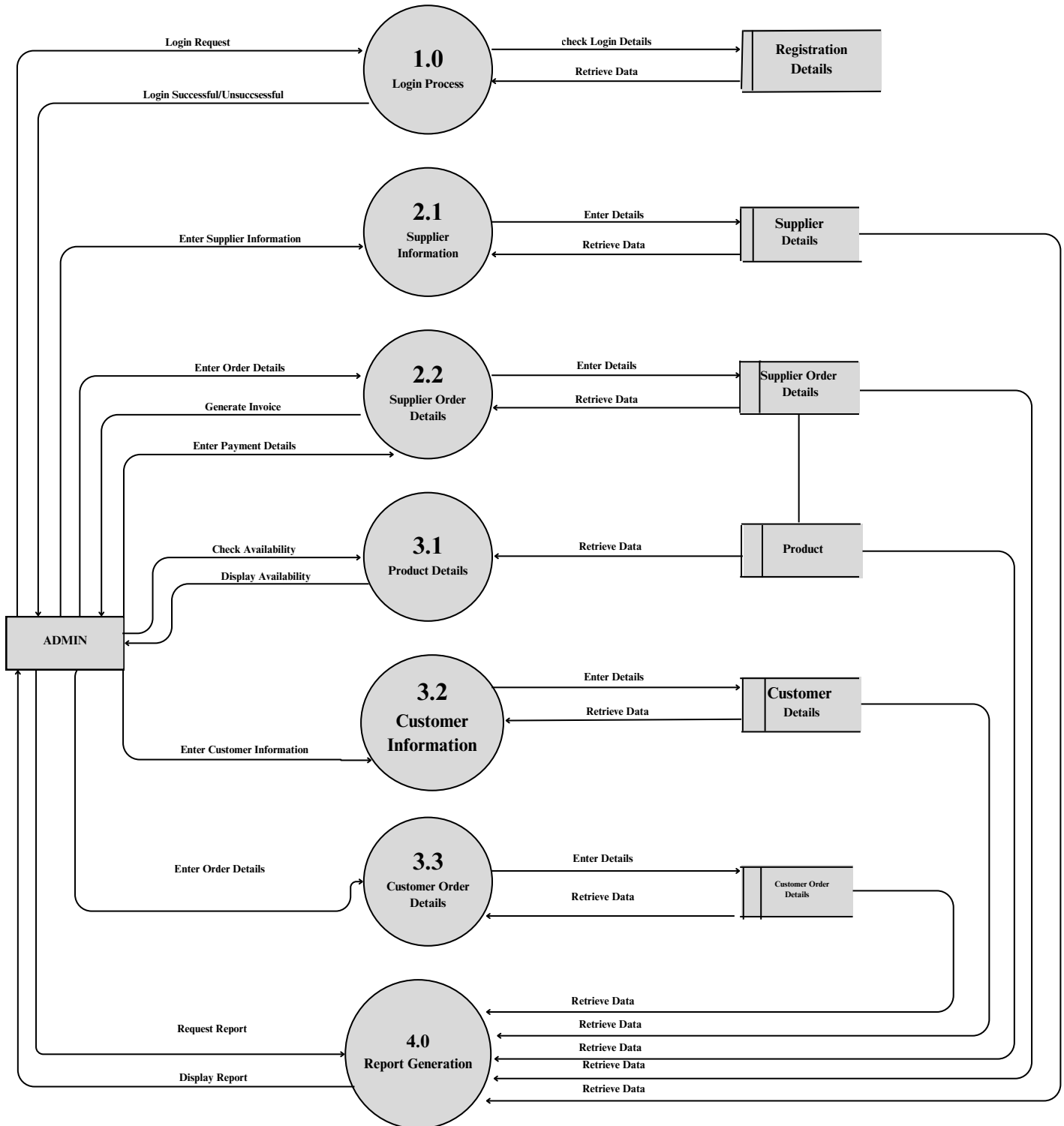
0- Level DFD



1-level DFD



2-level DFD



DATABASE DESIGN

Project Name: Fruit Inventory Management System

1.Table Name: registration_details

It store the registration details.

Sr.no	Field Name	Expansion Name	Field Type	Null/Not Null	Description
1	name	name	varchar (50)	not null	It will store the name of the user.
2	user name	user name	varchar (50)	not null	It will store the user name.
3	password	password	varchar (50)	not null	It will store the password.

2. Table Name: **supplier_order**

It store the supplier order

Sr.no	Field Name	Expansion Name	Field Type	Null/Not Null	Description
1	s_o_id	supplier order id	int	not null	It is a primary key and used to store the supplier order id. It will auto generate.
2	s_id	supplier id	int	not null	It is used to store supplier id.
3	date	date	date	not null	It is used to store the date.
4	total	total	int	not null	It is used to store the total amount of supplier order.

3.Table Name: supplier_personal_details

It store the supplier's personal details.

Sr.no	Field Name	Expansion Name	Field Type	Null/Not Null	Description
1	s_id	supplier id	int	not null	It is a primary key and used to store the supplier's id. It will auto generate.
2	s_name	supplier name	varchar (50)	not null	It is used to store the supplier's name.
3	state	state	varchar (50)	not null	It is used to store the supplier's state.
4	city	city	varchar (50)	not null	It is used to store the supplier's city.
5	district	district	varchar (50)	null	It is used to store the supplier's district
6	address	address	varchar (100)	null	It is used to store the supplier's address.
7	contact	contact	nchar (10)	not null	It is used to store the supplier's contact number.

4. Table Name: supplier_order_details

It store the supplier order details.

Sr.no	Field Name	Expansion Name	Field Type	Null/Not Null	Description
1	o_id	order id	int	not null	It is used to store order id. It will auto generate.
2	s_name	supplier name	varchar (50)	not null	It is used to store name of supplier.
3	f_name	fruit name	varchar (50)	not null	It is used to store the fruits name.
4	variety	variety	varchar (50)	not null	It is used to store the fruits variety
5	quant	quantity	int	not null	It is used to store the quantity of a fruit.
6	unit	unit	varchar (10)	not null	It is used to store the unit of fruit
7	price	price	int	not null	It is used to store the Price of fruit.

5. Table Name: customer_order

It store the customer's order .

Sr.no	Field Name	Expansion Name	Field Type	Null/Not Null	Description
1	c_o_id	customer order id	int	not null	It is used to store the customer order id. It will auto generate.
2	c_id	customer id	int	not null	It is used to store the customer id.
3	date	date	date	not null	It is used to store the date.
4	total	total	int	not null	It is used to store the total amount of customer order.

6.Table Name: customer_personal_details

It store the customer's personal details

Sr.no	Field Name	Expansion Name	Field Type	Null/Not Null	Description
1	c_id	customer id	int	not null	It is a primary key and it is used to store customer's id. It will auto generate.
2	c_name	customer name	varchar (50)	not null	It is used to store customer name.
3	state	state	varchar (50)	not null	It is used to store the customer's state
4	city	city	varchar (50)	not null	It is used to store the customer's city
5	district	district	varchar (50)	null	It is used to store the customer's district.
6	address	address	varchar (100)	null	It is used to store the customer's address
7	contact	contact	nchar (10)	not null	It is used to store the customer's contact number

7.Table Name: customer_order_details

It store the customer's order detail.

Sr.no	Field Name	Expansion Name	Field Type	Null/Not null	Description
1	o_id	order id	int	not null	It is used to store order id. It will auto generate
2	c_name	customer name	varchar (50)	not null	It is used to store name of customer.
3	f_name	fruit name	varchar (50)	not null	It is used to store fruit name.
4	variety	variety	varchar (50)	not null	It is used to store the variety of a fruit
5	quant	quantity	int	not null	It is used to store a quantity of a fruit.
6	unit	unit	varchar (50)	not null	It is used to store the unit of fruit.
7	price	price	int	not null	It is used to store the price of fruit.

8.Table Name: supplier_payment_details

It store the payment details of supplier.

Sr. no	Field Name	Expansion Name	Field Type	Null/ Not Null	Description
1	p_id	payment id	int	not null	It is used to store payment id. it will auto generate.
2	s_name	supplier name	varchar (50)	not null	It is used to store supplier name
3	date	date	int	not null	It is used to store date of transaction
4	amount	amount	int	not null	It will store amount paying to supplier.
5	transaction_mode	transaction mode	varchar (50)	not null	it will store mode of transaction online / cash
6	account_holder_name	account holder name	varchar (50)	not null	it will store the name of account holder .
7	contact	contact	nchar (10)	null	It is used to store the supplier's contact number.

9. Table Name: product_details
It stores the details of the product.

Sr.no	Field Name	Expansion Name	Field Type	Null/Not Null	Description
1	f_id	fruit id	int	not null	It is a primary key and used to store the fruit id. It will auto generate.
2	f_name	fruit name	varchar (50)	not null	It is used to store the fruit name.
3	variety	variety	varchar (50)	not null	It is used to store the variety of a fruit.
4	quant	quantity	int	not null	It is used to store the total quantity of product.

**INPUT OUTPUT SCREENS
TEST CASES
AND CODING**

Registration Form



A screenshot of a web browser window titled "Registration Form". The page has a red header with the text "Registration Information". Below the header, on the left, is a vibrant image of various fruits including grapes, kiwi, orange, banana, and papaya. On the right, there is a registration form with three input fields: "Name :", "Username :", and "Password :". Below the "Password :" field is a checkbox labeled "Show Password". At the bottom right of the form is a red button labeled "SAVE".



A screenshot of the same "Registration Form" web browser window. In this state, the "Name" and "Username" input fields are empty, while the "Password" field contains some text. A small error message dialog box is displayed in the foreground, titled "Suggestion", with a yellow warning icon and the text "Please fill all required fields.". The dialog box has "OK" and "SAVE" buttons. The "SAVE" button on the form is highlighted with a red border.

Registration From Coding

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace DNJ_Inventory_System
{
    public partial class login_details : Form
    {
        public login_details()
        {
            InitializeComponent();

            SqlConnection con = new SqlConnection(@"Data Source=LAPTOP-7O11E2P5\SQLEXPRESS;Initial
Catalog=dnjsystem;User ID=tanya;Password=1234;Encrypt=True;TrustServerCertificate=True");
            private void login_details_Load(object sender, EventArgs e)
            {
                check_pass.CheckedChanged += new EventHandler(check_pass_CheckedChanged);
                MaximizeBox = false;
            }
            private void text_Username_KeyPress(object sender, KeyPressEventArgs e)
            {
                if (!char.IsControl(e.KeyChar) && !char.IsLetter(e.KeyChar))
                {
                    e.Handled = true;
                }
            }
            private void text_FullName_KeyPress(object sender, KeyPressEventArgs e)
            {
                if (!char.IsControl(e.KeyChar) && !char.IsLetter(e.KeyChar)
&& !char.IsWhiteSpace(e.KeyChar))
                {
                    e.Handled = true;
                }
            }
        }
    }
}
```

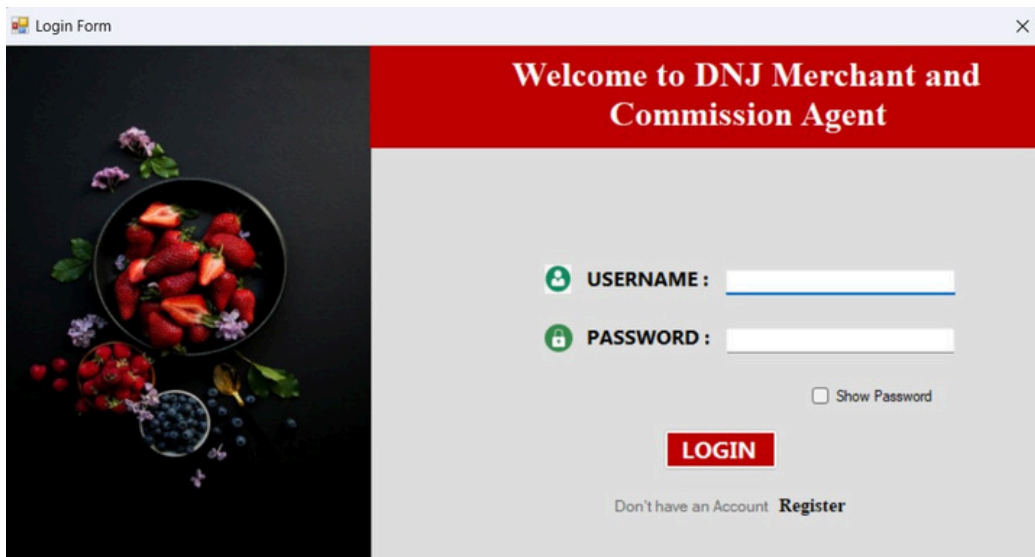
```

private void btn_save_Click(object sender, EventArgs e)
{
    //con.Open();
    SqlCommand comcheckid = new SqlCommand("SELECT username FROM registration_details WHERE
username='" + text_Username.Text + "'", con);
    SqlDataAdapter sd = new SqlDataAdapter(comcheckid);
    DataTable dt = new DataTable();
    sd.Fill(dt);
    if (dt.Rows.Count > 0)
    {
        MessageBox.Show("Username is already exist", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Warning);
    }
    else
    {
        if (text_FullName.Text != "" && text_Username.Text != "" && text_password.Text != "")
        {
            String str = @"Data Source=LAPTOP-7O11E2P5\SQLEXPRESS;Initial
Catalog=dnjsystem;User ID=tanya;Password=1234;Encrypt=True;TrustServerCertificate=True";
            SqlConnection con = new SqlConnection(str);
            con.Open();
            SqlCommand cmd1 = new SqlCommand(@"INSERT INTO registration_details
(name,username,password) VALUES('" + text_FullName.Text + "','" + text_Username.Text + "','" +
text_password.Text + "')", con);
            cmd1.ExecuteNonQuery();
            MessageBox.Show("Data saved Successfully.");
            con.Close();
            LoginForm l1 = new LoginForm();
            l1.Show();
            this.Hide();
        }
        else if (text_FullName.Text.Trim() == "" || text_Username.Text.Trim() == "" ||
text_password.Text.Trim() == "")
        {
            MessageBox.Show("Please fill all required fields.", "Suggestion", MessageBoxButtons.OK,
            MessageBoxIcon.Warning);
        }
    }
}

private void check_pass_CheckedChanged(object sender, EventArgs e)
{
    text_password.PasswordChar = check_pass.Checked ? '\0' : '*';
}
}
}

```

Login Form



This screenshot shows the initial login form for DNJ Merchant and Commission Agent. The window has a title bar "Login Form" and a close button. The main content area is split into two sections. On the left is a decorative image of a bowl of strawberries and blueberries. On the right is a red header with the text "Welcome to DNJ Merchant and Commission Agent". Below the header is a login form with two input fields: "USERNAME:" and "PASSWORD:". The "PASSWORD:" field has a "Show Password" checkbox. Below the fields is a red "LOGIN" button. At the bottom, there is a link "Don't have an Account Register".

Login Form

Welcome to DNJ Merchant and Commission Agent

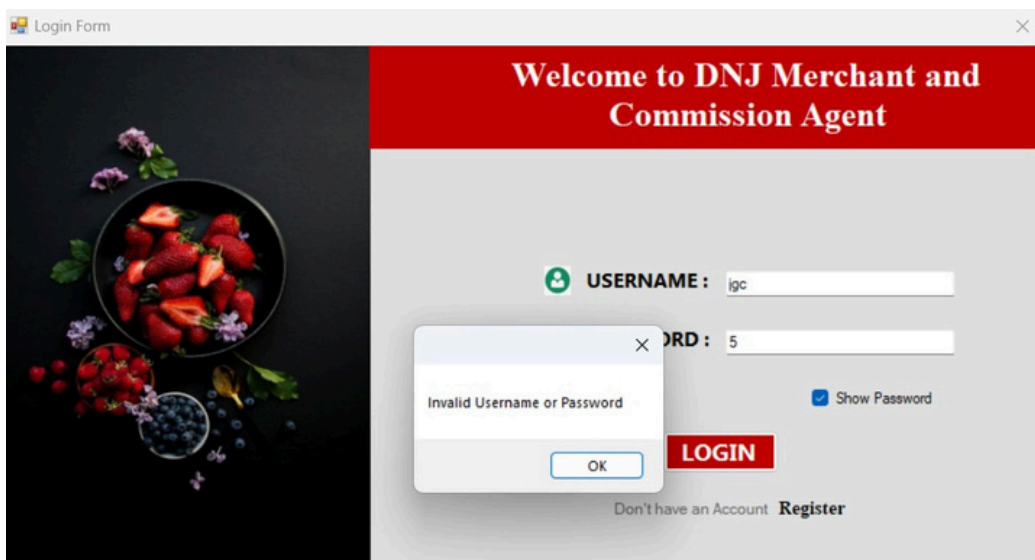
USERNAME:

PASSWORD:

☐ Show Password

LOGIN

Don't have an Account [Register](#)



This screenshot shows the login form after an attempt to log in with the username "igc" and password "5". An error message dialog box is displayed in the foreground, stating "Invalid Username or Password". The dialog box has a close button and an "OK" button. The background form shows the "USERNAME:" field with "igc" and the "PASSWORD:" field with "5". The "Show Password" checkbox is now checked. The "LOGIN" button is still visible, along with the "Don't have an Account Register" link.

Login Form

Welcome to DNJ Merchant and Commission Agent

USERNAME:

PASSWORD:

☒ Show Password

LOGIN

Don't have an Account [Register](#)

Invalid Username or Password

OK

Login Form Coding

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace DNJ_Inventory_System
{
    public partial class LoginForm : Form
    {
        //connection establish//
        public LoginForm()
        {
            InitializeComponent();
        }
        string Conn = (@"Data Source=LAPTOP-7O11E2P5\SQLEXPRESS;Initial Catalog=dnjsystem;User
ID=tanya;Password=1234;Encrypt=True;TrustServerCertificate=True");

        private void btn_2_Click(object sender, EventArgs e)
        {
        }

        private void btn_register_Click(object sender, EventArgs e)
        {
            login_details e1 = new login_details();
            e1.Show();
            this.Hide();
        }

        private void LoginForm_Load(object sender, EventArgs e)
        {
            // Subscribe to the CheckBox's CheckedChanged event
            check_pass.CheckedChanged += new EventHandler(check_pass_CheckedChanged);
            MaximizeBox = false;
        }

        private void text_username_KeyDown(object sender, KeyEventArgs e)
        {
            if (e.KeyCode == Keys.Enter)
            {
                text_password.Focus();
            }
        }
    }
}
```

```

private void text_password_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        btn_login.Focus();
    }
}

private void btn_login_Enter(object sender, EventArgs e)
{
    try
    {
        if (text_username.Text == "" && text_password.Text == "")
        {
            MessageBox.Show("Please Enter Username and password");
        }
    }
    else
    {
        SqlConnection con = new SqlConnection(Conn);
        SqlCommand cmd = new SqlCommand("select * from registration_details where username=@username and password=@password", con);
        cmd.Parameters.AddWithValue("@username", text_username.Text);
        cmd.Parameters.AddWithValue("@password", text_password.Text);
        con.Open();
        SqlDataAdapter adpt = new SqlDataAdapter(cmd);
        DataSet ds = new DataSet();
        adpt.Fill(ds);
        con.Close();
        int count = ds.Tables[0].Rows.Count;
        if (count == 1)
        {
            MessageBox.Show("Login Successfull");
            this.Hide();
            Dashboard d1 = new Dashboard();
            d1.Show();
        }
        else
        {
            MessageBox.Show("Invalid Username or Password");
        }
    }
}

catch (Exception ex)
{
    MessageBox.Show(ex.Message);
}
}

```

```

private void btn_login_Click(object sender, EventArgs e)
{
    string username = text_username.Text;
    string password = text_password.Text;

    if (username.Equals(username, StringComparison.Ordinal) &&
        password.Equals(password, StringComparison.Ordinal))
    {
        try
        {
            if (text_username.Text == "" && text_password.Text == "")
            {
                MessageBox.Show("Please Enter the Username and Password");
            }
            else
            {
                string queryText = "select Count(*) from registration_details" + "where
username=@username and password=@password COLLATE SQL_Latin1_general_CP1_CS_AS";
                SqlConnection cn = new SqlConnection(@"Data Source=LAPTOP-
7O11E2P5\SQLEXPRESS;Initial Catalog=dnjsystem;User
ID=tanya;Password=1234;Encrypt=True;TrustServerCertificate=True");
                SqlCommand cmd = new SqlCommand(queryText, cn);
                {
                    cn.Open();
                    cmd.Parameters.AddWithValue("@username", text_username.Text);
                    cmd.Parameters.AddWithValue("@password", text_password.Text);
                    int result = Convert.ToInt32(cmd.ExecuteScalar());
                    if (result > 0)
                    {
                        MessageBox.Show("Login Successfull");
                        this.Hide();
                        Dashboard d1 = new Dashboard();
                        d1.Show();
                    }
                    else
                    {
                        MessageBox.Show("Invalid Username Or Password");
                    }
                    check_pass.Checked = false;
                }
            }
        }
        catch (Exception ex)
        {
            MessageBox.Show(ex.Message);
        }
    }
}

```

```
else
{
    MessageBox.Show("Invalid Username Or Password");
}

}

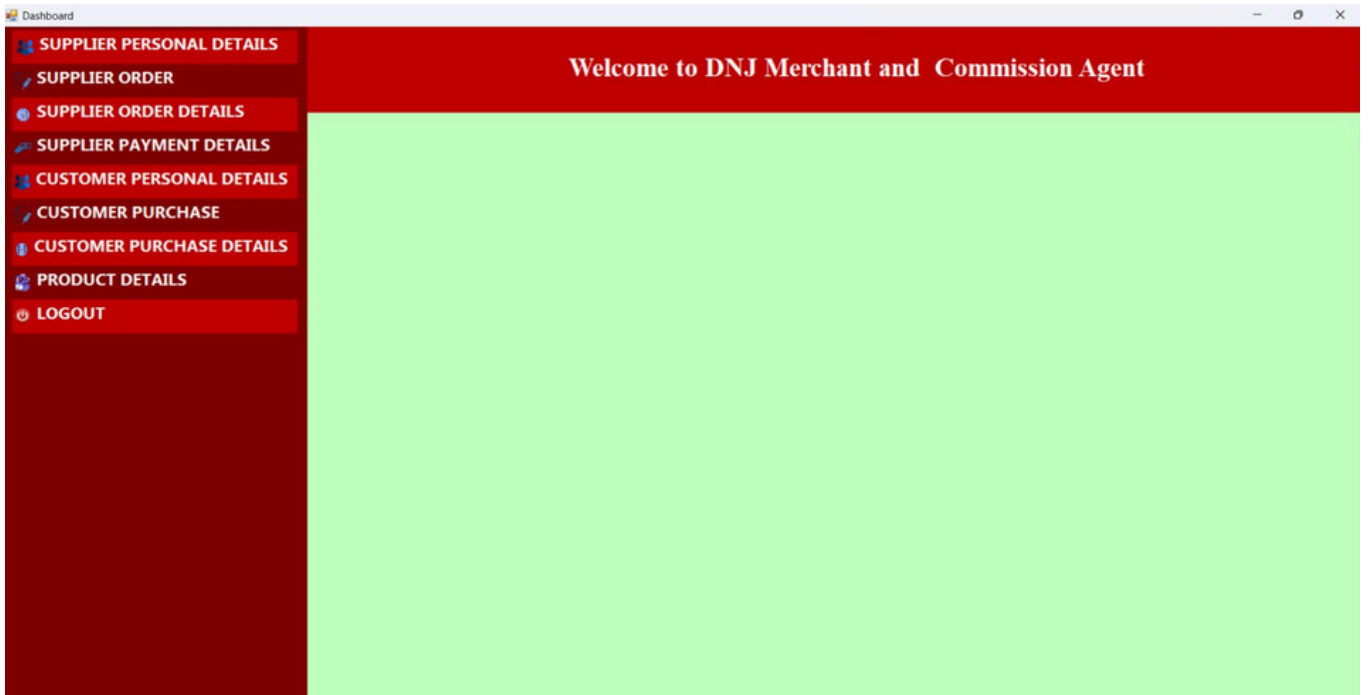
private void check_pass_CheckedChanged(object sender, EventArgs e)
{
    text_password.PasswordChar = check_pass.Checked ? '\0' : '*';
}

private void label1_Click(object sender, EventArgs e)
{
}

}

}
```

Dashboard



Dashboard Coding

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using System.Data.SqlClient;
using System.Xml.Linq;

namespace DNJ_Inventory_System
{
    public partial class Dashboard : Form
    {
        public Dashboard()
        {
            InitializeComponent();
            this.Size = new Size(800, 600); // Set your desired size
            this.Visible = true;
        }

        private void Dashboard_Load(object sender, EventArgs e)
        {
        }

        private void sUPPLIERPERSONALDETAILSToolStripMenuItem_Click(object sender, EventArgs e)
        {
            personal_details sp1 = new personal_details();
            sp1.Show();
        }

        private void sUPPLIERORDERToolStripMenuItem_Click(object sender, EventArgs e)
        {
            supplier_order so = new supplier_order();
            so.Show();
        }

        private void sUPPLIERORDERDETAILSToolStripMenuItem_Click(object sender, EventArgs e)
        {
            supplier_order_details sod1 = new supplier_order_details();
            sod1.Show();
        }
    }
}
```

```

private void sUPPLIERPAYMENTDETAILSToolStripMenuItem_Click(object sender, EventArgs e)
{
    supplier_payment_details spd = new supplier_payment_details();
    spd.Show();
}

private void cUSTOMERPERSONALDETAILSToolStripMenuItem_Click(object sender, EventArgs e)
{
    customer_personal_details cpd = new customer_personal_details();
    cpd.Show();
}

private void cUSTOMERORDERToolStripMenuItem_Click(object sender, EventArgs e)
{
    customer_order co = new customer_order();
    co.Show();
}

private void cUSTOMERORDERDETAILSToolStripMenuItem_Click(object sender, EventArgs e)
{
    customer_order_details cod = new customer_order_details();
    cod.Show();
}

private void pRODUCTDETAILSToolStripMenuItem_Click(object sender, EventArgs e)
{
    product_details pd = new product_details();
    pd.Show();
}

private void LOGOUTToolStripMenuItem_Click_1(object sender, EventArgs e)
{
    LoginForm l1 = new LoginForm();
    l1.Show();
}

private void label1_Click(object sender, EventArgs e)
{
}

private void supplierDetailsToolStripMenuItem_Click(object sender, EventArgs e)
{
}

private void orderDetailsToolStripMenuItem_Click(object sender, EventArgs e)
{
}

private void eXITToolStripMenuItem1_Click(object sender, EventArgs e)
{
}

```

```

private void LOGOUTToolStripMenuItem1_Click(object sender, EventArgs e)
{

}

private void supplierToolStripMenuItem_Click(object sender, EventArgs e)
{

}

private void LOGOUTToolStripMenuItem_Click(object sender, EventArgs e)
{

}

private void eXITToolStripMenuItem_Click(object sender, EventArgs e)
{

}

private void menuStrip3_ItemClicked(object sender, ToolStripItemClickedEventArgs e)
{

}

private void customerDetailsToolStripMenuItem_Click(object sender, EventArgs e)
{

}

private void orderDetailsToolStripMenuItem1_Click(object sender, EventArgs e)
{

}

private void orderToolStripMenuItem_Click(object sender, EventArgs e)
{

}

private void purchasedDetailsToolStripMenuItem_Click(object sender, EventArgs e)
{

)

private void supplierPaymentToolStripMenuItem_Click(object sender, EventArgs e)
{

}

```

```
private void proToolStripMenuItem_Click(object sender, EventArgs e)
{
}

private void pAYMENTToolStripMenuItem_Click(object sender, EventArgs e)
{
}

private void pictureBox1_Click(object sender, EventArgs e)
{
}

private void panell1_Paint(object sender, PaintEventArgs e)
{
}
}
```

Supplier Personal Details Form

Supplier Personal Details

SEARCH :

Supplier Id : 00001
Name :
State :
City/Village :
District :
Address :
Contact :

	S_ID	Name	State
*			

SAVE UPDATE REFRESH

Supplier Personal Details

SEARCH :

Supplier Id : 00001
Name : Tanya
State : Maharashtra
City/Village : xyx
District : Raigad
Address : hyk
Contact : 86135

	S_ID	Name	State
*			

Error

! Contact number must be 10 digits and start with 6, 7, 8, or 9

OK

SAVE UPDATE REFRESH

Supplier Personal Details Form Coding

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Threading.Tasks;
using System.Windows.Forms;
using static System.Windows.Forms.VisualStyles.VisualStyleElement;

namespace DNJ_Inventory_System
{
    public partial class personal_details : Form
    {
        public personal_details()
        {
            InitializeComponent();
        }
        private const Keys k_copy = Keys.Control | Keys.C;
        private const Keys k_paste = Keys.Control | Keys.V;
        protected override bool ProcessCmdKey(ref Message msg, Keys keyData) // to disable copy paste
        {
            if ((keyData == k_copy) || (keyData == k_paste))
            {
                return true;
            }
            else
            {
                return base.ProcessCmdKey(ref msg, keyData);
            }
        }

        SqlConnection con = new SqlConnection(@"Data Source=LAPTOP-7O11E2P5\SQLEXPRESS;Initial
Catalog=dnjssystem;User ID=tanya;Password=1234;Encrypt=True;TrustServerCertificate=True");
        SqlDataAdapter adapter;
        DataTable dt;
        private object textBoxValueToSearch;

        public void Getsupplierid()
        {
            string supid;
            string query = "select s_id from s_personal_details order by s_id Desc";
            con.Open();
            SqlCommand cmd = new SqlCommand(query, con);
            SqlDataReader dr = cmd.ExecuteReader();
            if (dr.Read())
            {

```

```

int idr = int.Parse(dr[0].ToString()) + 1;
    supid = idr.ToString("00000");
}
else if (Convert.IsDBNull(dr))
{
    supid = ("00001");

}
else
{
    supid = ("00001");
}
con.Close();
text_id.Text = supid.ToString();
}

private void personal_details_Load(object sender, EventArgs e)
{
    Getsupplierid(); // Get and display the supplier ID

    try
    {
        using (SqlConnection con = new SqlConnection(@"Data Source=LAPTOP-7O11E2P5\SQLEXPRESS;Initial
Catalog=dnjsystem;User ID=tanya;Password=1234;Encrypt=True;TrustServerCertificate=True"))
        {
            adapter = new SqlDataAdapter("SELECT * FROM s_personal_details", con);
            dt = new DataTable(); // Assign to the class-level dt variable

            con.Open();

            // Move the data retrieval code here
            adapter.Fill(dt);
            s_personal_details_dataGridView.DataSource = dt;

            // Also, fill the dataset here
            this.s_personal_detailsTableAdapter.Fill(this.dnjsystemDataSet.s_personal_details);
        }

        Getsupplierid(); // Get and display the supplier ID
    }
    catch (Exception)
    {
        // Handle exceptions here
    }
}

// Method to clear all textboxes except ID
private void ClearTextBoxesExceptID()
{
    text_name.Text = "";
    text_state.Text = "";
    text_city.Text = "";
}

```

```

text_district.Text = "";
    text_address.Text = "";
    text_contact.Text = "";
}
private void Form1_Load(object sender, EventArgs e)
{
    // TODO: This line of code loads data into the 'dnjsystemDataSet.staff_details' table. You can move, or remove it,
as needed.
    this.s_personal_detailsTableAdapter.Fill(this.dnjsystemDataSet.s_personal_details);
}

private void btn_update_Click(object sender, EventArgs e)
{
    string id = text_id.Text;
    bool idExists = false;

    foreach (DataGridViewRow row in s_personal_details_dataGridView.Rows)
    {
        if (row.Cells[0].Value != null && row.Cells[0].Value.ToString() == id)
        {
            idExists = true;
        }
    }

    if (idExists)
    {
        if (text_id.Text != "" && text_name.Text != "" && text_state.Text != "" && text_city.Text != "" &&
text_district.Text != "" && text_contact.Text != "")
        {
            string query = "UPDATE s_personal_details set
s_name=@s_name,state=@state,city=@city,district=@district,address=@address,contact=@contact WHERE
s_id=@s_id";
            SqlCommand cmd = new SqlCommand(query, con);
            cmd.Parameters.AddWithValue("@s_id", Convert.ToInt32(text_id.Text));
            cmd.Parameters.AddWithValue("@s_name", (text_name.Text));
            cmd.Parameters.AddWithValue("@state", (text_state.Text));
            cmd.Parameters.AddWithValue("@city", (text_city.Text));
            cmd.Parameters.AddWithValue("@district", (text_district.Text));
            cmd.Parameters.AddWithValue("@address", string.IsNullOrEmpty(text_address.Text) ? DBNull.Value :
(object)text_address.Text); // Handle empty address
            cmd.Parameters.AddWithValue("@contact", (text_contact.Text));

            con.Open();
            cmd.ExecuteNonQuery();
            con.Close();
            display_data();
            text_id.Text = " ";
            text_name.Text = " ";
            text_state.Text = "";
            text_city.Text = " ";
            text_district.Text = " ";
            text_address.Text = " ";
            text_contact.Text = " ";

```

```

MessageBox.Show("Data Successfully Updated", "Empty", MessageBoxButtons.OK, MessageBoxIcon.Warning);
    Getsupplierid();
}
else
{
    MessageBox.Show("Please select row which you want to update", "Empty", MessageBoxButtons.OK,
MessageBoxIcon.Warning);
}
}
else
{
    MessageBox.Show("ID Does Not Exist.");
}
}

private void display_data()
{
    string query = "SELECT * FROM s_personal_details";
    con.Open();
    SqlDataAdapter da = new SqlDataAdapter(query, con);
    DataTable dt = new DataTable();
    da.Fill(dt);
    s_personal_details_dataGridView.DataSource = dt;
    con.Close();
}
private void btn_save_Click(object sender, EventArgs e)
{
    SqlCommand comcheckid = new SqlCommand("select s_id from s_personal_details where s_id=" + text_id.Text +
"", con);
    SqlDataAdapter sd = new SqlDataAdapter(comcheckid);
    DataTable dt = new DataTable();
    sd.Fill(dt);

    if (dt.Rows.Count > 0)
    {
        MessageBox.Show("Id is already exist", "Error", MessageBoxButtons.OK, MessageBoxIcon.Warning);
    }
    else
    {
        if (text_id.Text != "" && text_name.Text != "" && text_state.Text != "" && text_city.Text != "" &&
text_district.Text != "" && text_contact.Text != "")
        {
            // Validate contact number length and format
            if (text_contact.Text.Length != 10 || !text_contact.Text.StartsWith("6") && !text_contact.Text.StartsWith("7")
&& !text_contact.Text.StartsWith("8") && !text_contact.Text.StartsWith("9"))
            {
                MessageBox.Show("Contact number must be 10 digits and start with 6, 7, 8, or 9", "Error",
MessageBoxButtons.OK, MessageBoxIcon.Warning);
                return; // Exit the method if contact number is not valid
            }
        }
    }
}

```

```

// Clear error provider
errorProvider1.Clear();

// Insert data into database
SqlCommand cmd1 = new SqlCommand(@"INSERT INTO s_personal_details (s_id, s_name, state, city,
district, address, contact) VALUES(@s_id, @s_name, @state, @city, @district, @address, @contact)", con);
cmd1.Parameters.AddWithValue("@s_id", text_id.Text);
cmd1.Parameters.AddWithValue("@s_name", text_name.Text);
cmd1.Parameters.AddWithValue("@state", text_state.Text);
cmd1.Parameters.AddWithValue("@city", text_city.Text);
cmd1.Parameters.AddWithValue("@district", text_district.Text);

// Check if address is provided, if not, set it to DBNull.Value
if (string.IsNullOrEmpty(text_address.Text))
{
    cmd1.Parameters.AddWithValue("@address", DBNull.Value);
}
else
{
    cmd1.Parameters.AddWithValue("@address", text_address.Text);
}

cmd1.Parameters.AddWithValue("@contact", text_contact.Text);

con.Open();
cmd1.ExecuteNonQuery();
con.Close();

MessageBox.Show("Data saved Successfully.");
display_data();

text_name.Text = "";
text_state.Text = "";
text_city.Text = "";
text_district.Text = "";
text_address.Text = "";
text_contact.Text = "";
Getsupplierid();
}
else
{
    MessageBox.Show("Please Fill Empty Fields", "Suggestion", MessageBoxButtons.OK,
    MessageBoxIcon.Warning);
}
}
}

private void text_contact_TextChanged(object sender, EventArgs e)
{
    if (text_contact.TextLength == 10)
    {
        text_contact.ForeColor = Color.Black;
    }
}

```

```

else
{
    text_contact.ForeColor = Color.Red;
}
}
private void text_contact_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar) && !char.IsDigit(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show("A phone Number cannot contains Letter.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        text_contact.Focus();
    }
}
private void text_name_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar) && !char.IsLetter(e.KeyChar) && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;

        MessageBox.Show("A Name cannot contains Number.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        text_name.Focus();
    }
}
private void text_id_TextChanged(object sender, EventArgs e)
{
    text_name.Text = "";
}
private void text_supname_KeyPress(object sender, KeyPressEventArgs e)
{
}
private void text_supid_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsDigit(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show("Supplier ID cannot contain Letter.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
    }
}

private void text_supname_TextChanged(object sender, EventArgs e)
{
}

```

```

private void text_id_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_name.Focus();
    }
}

private void text_name_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_state.Focus();
    }
}

private void text_state_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_city.Focus();
    }
}

private void text_city_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_district.Focus();
    }
}

private void text_district_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_address.Focus();
    }
}

private void text_address_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_contact.Focus();
    }
}

private void text_name_KeyPress1(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsLetter(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {

```

```

e.Handled = true;
    MessageBox.Show("A Name cannot contains Number.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    text_name.Focus();

}
}

private void s_personal_details_dataGridView_CellEnter(object sender, DataGridViewCellEventArgs e)
{
    text_id.Text = s_personal_details_dataGridView.CurrentRow.Cells[0].Value.ToString();
    text_name.Text = s_personal_details_dataGridView.CurrentRow.Cells[1].Value.ToString();
    text_state.Text = s_personal_details_dataGridView.CurrentRow.Cells[2].Value.ToString();
    text_city.Text = s_personal_details_dataGridView.CurrentRow.Cells[3].Value.ToString();
    text_district.Text = s_personal_details_dataGridView.CurrentRow.Cells[4].Value.ToString();
    text_address.Text = s_personal_details_dataGridView.CurrentRow.Cells[5].Value.ToString();
    text_contact.Text = s_personal_details_dataGridView.CurrentRow.Cells[6].Value.ToString();
}

private void btn_refresh_Click(object sender, EventArgs e)
{
    display_data();
    text_name.Text = "";
    text_state.Text = "";
    text_city.Text = "";
    text_district.Text = "";
    text_address.Text = "";
    text_contact.Text = "";
    Getsupplierid();
}

private void text_contact_Leave(object sender, EventArgs e)
{
    Regex re = new Regex(@"^[6789]\d*$");
    if (re.IsMatch(text_contact.Text.Trim()) == false || text_contact.Text.Length != 10)
    {
        errorProvider2.SetError(text_contact, "Invalid Mobile Number");
        text_contact.Focus();
    }
    else
    {
        errorProvider2.SetError(text_contact, string.Empty);
    }
}

private void s_personal_details_dataGridView_CellMouseMove(object sender, DataGridViewCellEventArgs e)
{
}

private void s_personal_details_dataGridView_CellClick(object sender, DataGridViewCellEventArgs e)
{
}

```

```

private void text_city_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsLetter(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show("A City/Village cannot contains Number.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        text_city.Focus();
    }
}

private void text_state_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsLetter(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show("A State cannot contains Number.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        text_state.Focus();
    }
}

private void text_district_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsLetter(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show("A District cannot contains Number.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        text_district.Focus();
    }
}

private void panell1_Paint(object sender, PaintEventArgs e)
{
}

private void text_state_TextChanged(object sender, EventArgs e)
{
}

public void searchData(string valuetosearch)
{
}

```

```

private void button1_Click(object sender, EventArgs e)
{
}

private void s_personal_details_dataGridView_CellContentClick(object sender, DataGridViewCellEventArgs e)
{
}

private void text_search_KeyPress(object sender, KeyPressEventArgs e)
{
}

// Method to clear textboxes
private void ClearTextBoxes()
{
    text_name.Text = "";
    text_state.Text = "";
    text_city.Text = "";
    text_district.Text = "";
    text_address.Text = "";
    text_contact.Text = "";
}

private void text_search_TextChanged(object sender, EventArgs e)
{
    // Get the search keyword from the TextBox
    string keyword = text_search.Text;

    // Check if the keyword is not empty
    if (!string.IsNullOrEmpty(keyword))
    {
        // Filter the DataGridView based on the keyword
        DataView dv = dt.DefaultView;
        string filterExpression = $"s_name LIKE '%{keyword}%' OR state LIKE '%{keyword}%' OR city LIKE '%{keyword}%' OR district LIKE '%{keyword}%' OR address LIKE '%{keyword}%' OR Convert(contact, 'System.String') LIKE '%{keyword}%'";
        Console.WriteLine("Filter Expression: " + filterExpression); // Debugging
        dv.RowFilter = filterExpression;
        Console.WriteLine("Filtered Rows Count: " + dv.Count); // Debugging
        s_personal_details_dataGridView.DataSource = dv;
    }
    else
    {
        // If the search TextBox is empty, show all data
        DataView dv = dt.DefaultView;
        dv.RowFilter = ""; // Clear any existing filter
        s_personal_details_dataGridView.DataSource = dv;
    }
}

```

```
// Clear the textboxes
    ClearTextBoxes();
}

private void s_id_Click(object sender, EventArgs e)
{

}

private void s_name_Click(object sender, EventArgs e)
{

}
}
}
```

Supplier Order Form

supplier_order

Supplier Order

FROM : 20 April 2024

TO : 20 April 2024

SEARCH

Supplier Order Id : 00001

Supplier Id : 00001

Date : 20 April 2024

Total :

S_O_ID	S_ID	Date
*		

SAVE

UPDATE

REFRESH

TOTAL 0

CLEAR

supplier_order

Supplier Order

FROM : 20 April 2024

TO : 01 April 2024

SEARCH

Supplier Order Id : 00001

Supplier Id : 00001

Date : 20 April 2024

Total : 200

Validation Error

End date must be after the start date.

OK

S_O_ID	S_ID	Date

SAVE

UPDATE

REFRESH

TOTAL 0

CLEAR

Supplier Order Form Coding

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace DNJ_Inventory_System
{
    public partial class supplier_order : Form
    {
        private const Keys k_copy = Keys.Control | Keys.C;
        private const Keys k_paste = Keys.Control | Keys.V;

        SqlConnection con = new SqlConnection(@"Data Source=LAPTOP-7O11E2P5\SQLEXPRESS;Initial
Catalog=dnjsystem;User ID=tanya;Password=1234;Encrypt=True;TrustServerCertificate=True");
        SqlDataAdapter adapter;
        DataTable dt;

        public supplier_order()
        {
            InitializeComponent();
        }
        protected override bool ProcessCmdKey(ref Message msg, Keys keyData)
        {
            if ((keyData == k_copy) || (keyData == k_paste))
            {
                return true;
            }
            else
            {
                return base.ProcessCmdKey(ref msg, keyData);
            }
        }
        private void Getsupplier_order_id()
        {
            string s_orderid;
            string query = "select s_o_id from s_order order by s_o_id Desc";
            con.Open();
            SqlCommand cmd = new SqlCommand(query, con);
            SqlDataReader dr = cmd.ExecuteReader();
            if (dr.Read())
            {
                int idr = int.Parse(dr[0].ToString()) + 1;
                s_orderid = idr.ToString("00000");
            }
        }
    }
}
```

```

else if (Convert.IsDBNull(dr))
{
    s_orderid = ("00001");

}
else
{
    s_orderid = ("00001");
}
con.Close();
text_s_order_id.Text = s_orderid.ToString();
}

public void Getsupplier_id()
{
    string supid;
    string query = "select s_id from s_order order by s_id Desc";
    con.Open();
    SqlCommand cmd = new SqlCommand(query, con);
    SqlDataReader dr = cmd.ExecuteReader();
    if (dr.Read())
    {
        int idr = int.Parse(dr[0].ToString()) + 1;
        supid = idr.ToString("00000");
    }
    else if (Convert.IsDBNull(dr))
    {
        supid = ("00001");

    }
    else
    {
        supid = ("00001");
    }
    con.Close();
    text_supplier_id.Text = supid.ToString();
}

private void supplier_order_Load(object sender, EventArgs e)
{
    Getsupplier_order_id();
    Getsupplier_id();
    try
    {
        using (SqlConnection con = new SqlConnection(@"Data Source=LAPTOP-7O11E2P5\SQLEXPRESS;Initial
Catalog=dnjsystem;User ID=tanya;Password=1234;Encrypt=True;TrustServerCertificate=True"))
        {
            adapter = new SqlDataAdapter("SELECT * FROM s_order", con);
            dt = new DataTable();

            con.Open();

```

```

// Move the data retrieval code here
adapter.Fill(dt);
s_order_dataGridView.DataSource = dt;

// Also, fill the dataset here
this.s_order_TableAdapter.Fill(this.dnjsystemDataSet.s_order);
// Now, set the SelectionMode and assign the event handler
s_order_dataGridView.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
s_order_dataGridView.SelectionChanged += s_order_dataGridView_SelectionChanged;
}

// Now, the rest of your initialization code
Getsupplier_order_id();
Getsupplier_id();

text_total.Text = " ";

}

catch (Exception)
{
    // Handle exceptions here
}

}

private void s_order_dataGridView_SelectionChanged(object sender, EventArgs e)
{
    if (s_order_dataGridView.SelectedRows.Count > 0)
    {
        int rowIndex = s_order_dataGridView.SelectedRows[0].Index;
        text_s_order_id.Text = s_order_dataGridView.Rows[rowIndex].Cells[0].Value.ToString();
        text_supplier_id.Text = s_order_dataGridView.Rows[rowIndex].Cells[1].Value.ToString();
        dateTimePicker1.Value = Convert.ToDateTime(s_order_dataGridView.Rows[rowIndex].Cells[2].Value);
        text_total.Text = s_order_dataGridView.Rows[rowIndex].Cells[3].Value.ToString();
    }
}

private void Form_Load(object sender, EventArgs e)
{
    // TODO: This line of code loads data into the 'dnjsystemDataSet.s_order' table. You can move, or remove it, as needed.
    this.s_order_TableAdapter.Fill(this.dnjsystemDataSet.s_order);
}

private void btn_update_Click(object sender, EventArgs e)
{
    string s_order_id = text_s_order_id.Text;
    string supplier_id = text_supplier_id.Text;

```

```

bool s_order_idExists = CheckIfIdExists(s_order_id);
bool supplier_idExists = CheckIfIdExists(supplier_id);

if (s_order_idExists)
{
    if (text_s_order_id.Text != "" && text_supplier_id.Text != "" && text_total.Text != "" && dateTimePicker1.Text
!= "")
    {
        string query = "UPDATE s_order SET s_id=@s_id, total=@total, date=@date WHERE s_o_id=@s_o_id";
        SqlCommand cmd = new SqlCommand(query, con);
        cmd.Parameters.AddWithValue("@s_o_id", Convert.ToInt32(s_order_id));
        cmd.Parameters.AddWithValue("@s_id", Convert.ToInt32(supplier_id));
        cmd.Parameters.AddWithValue("@total", Convert.ToInt32(text_total.Text));
        cmd.Parameters.AddWithValue("@date", dateTimePicker1.Text);

        con.Open();
        cmd.ExecuteNonQuery();
        con.Close();
        display_data();
        ClearTextBoxes();

        MessageBox.Show("Data Successfully Updated", "Success", MessageBoxButtons.OK,
        MessageBoxIcon.Information);
        Getsupplier_order_id();
        Getsupplier_id();
    }
    else
    {
        MessageBox.Show("Please select row which you want to update", "Empty", MessageBoxButtons.OK,
        MessageBoxIcon.Warning);
    }
}
else
{
    if (text_s_order_id.Text == "" || text_supplier_id.Text == "" || dateTimePicker1.Text == "" || text_total.Text == "")
    {
        MessageBox.Show("Please fill in all the required fields", "Empty Fields", MessageBoxButtons.OK,
        MessageBoxIcon.Warning);
    }
    else
    {
        MessageBox.Show("Order ID, does not exist.", "Not Found", MessageBoxButtons.OK,
        MessageBoxIcon.Warning);
    }
}
}

private bool CheckIfIdExists(string id)
{
    foreach (DataRow row in dt.Rows)
    {

```

```

if (row[0].ToString() == id)
    {
        return true;
    }
}
return false;
}
private void ClearTextBoxes()
{
    text_s_order_id.Text = "";
    text_supplier_id.Text = "";
    dateTimePicker1.Text = "";
    text_total.Text = "";
}
private void display_data()
{
    string query = "SELECT * FROM s_order";
    con.Open();
    SqlDataAdapter da = new SqlDataAdapter(query, con);
    dt.Clear();
    da.Fill(dt);
    s_order_dataGridView.DataSource = dt;
    con.Close();
}

private void btn_save_Click(object sender, EventArgs e)
{
    string order_id = text_s_order_id.Text;

    bool order_idExists = CheckIfIdExists(order_id, "s_o_id");
    bool supplier_idExists = CheckIfIdExists(text_supplier_id.Text, "s_id");

    if (order_idExists || supplier_idExists)
    {
        MessageBox.Show("Order ID, already exists", "Error", MessageBoxButtons.OK, MessageBoxIcon.Warning);
    }
    else
    {
        if (text_s_order_id.Text != "" && dateTimePicker1.Text != "" && text_total.Text != "")
        {
            try
            {
                string query = "INSERT INTO s_order VALUES(@s_o_id, @s_id, @date, @total)";
                SqlCommand cmd1 = new SqlCommand(query, con);
                cmd1.Parameters.AddWithValue("@s_o_id", Convert.ToInt32(text_s_order_id.Text));
                cmd1.Parameters.AddWithValue("@s_id", Convert.ToInt32(text_supplier_id.Text));
                cmd1.Parameters.AddWithValue("@date", dateTimePicker1.Text);
                cmd1.Parameters.AddWithValue("@total", Convert.ToInt32(text_total.Text));
                con.Open();
                cmd1.ExecuteNonQuery();
                MessageBox.Show("Data saved Successfully.", "Success", MessageBoxButtons.OK,
                MessageBoxIcon.Information);
                con.Close();
                display_data();
                ClearTextBoxes();
            }
            catch
            {
            }
        }
    }
}

```

```

catch (Exception ex)
{
    MessageBox.Show("Error: " + ex.Message, "Error", MessageBoxButtons.OK,
MessageBoxIcon.Error);
}
else
{
    MessageBox.Show("Please Fill Empty Fields", "Suggestion", MessageBoxButtons.OK,
MessageBoxIcon.Warning);
}
}
}
private bool CheckIfIdExists(string id, string columnName)
{
    foreach (DataRow row in dt.Rows)
    {
        if (row[columnName].ToString() == id)
        {
            return true;
        }
    }
    return false;
}

private void text_order_id_TextChanged(object sender, EventArgs e)
{
    dateTimePicker1.Text = "";
}

private void text_order_id_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        dateTimePicker1.Focus();
    }
}

private void text_order_id_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsDigit(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show(" Order ID cannot contains Letter.", "Error", MessageBoxButtons.OK,
MessageBoxIcon.Error);
    }
}

private void dateTimePicker1_KeyPress(object sender, KeyPressEventArgs e)
{

```

```

if (!char.IsControl(e.KeyChar)
    && !char.IsDigit(e.KeyChar)
    && !char.IsLetter(e.KeyChar)
    && !char.IsWhiteSpace(e.KeyChar))
{
    e.Handled = true;
    MessageBox.Show(" Please select date, typing anything is not allowed.", "Error", MessageBoxButtons.OK,
    MessageBoxIcon.Error);
}

private void text_total_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsDigit(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show(" Total cannot contains Letter.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
    // Allow only one decimal point
    if (e.KeyChar == '.' && (sender as TextBox).Text.Contains("."))
    {
        e.Handled = true;
        MessageBox.Show("total can only contain one decimal point.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
    }
}

private void dateTimePicker1_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_total.Focus();
    }
}

private void s_order_dataGridView_CellEnter(object sender, DataGridViewCellEventArgs e)
{
    if (s_order_dataGridView.SelectedRows.Count > 0)
    {
        int rowIndex = s_order_dataGridView.SelectedRows[0].Index;
        {
            text_s_order_id.Text = s_order_dataGridView.CurrentRow.Cells[0].Value.ToString();
            text_supplier_id.Text = s_order_dataGridView.CurrentRow.Cells[1].Value.ToString();
            dateTimePicker1.Text = s_order_dataGridView.CurrentRow.Cells[2].Value.ToString();
            text_total.Text = s_order_dataGridView.CurrentRow.Cells[3].Value.ToString();
        }
    }
}

```

```

private void btn_refresh_Click(object sender, EventArgs e)
{

    display_data();
    dateTimePicker1.Text = "";
    text_total.Text = "";

    Getsupplier_order_id();
    Getsupplier_id();

}

private void s_order_dataGridView_SelectionChanged_1(object sender, EventArgs e)
{

}

private void s_order_dataGridView_CellContentClick(object sender, DataGridViewCellEventArgs e)
{

}

private void SearchOrders(string searchTerm)
{
    s_order_dataGridView.ClearSelection();

    if (string.IsNullOrEmpty(searchTerm))
    {
        MessageBox.Show("Please enter a valid search term.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        return;
    }

    foreach (DataGridViewRow row in s_order_dataGridView.Rows)
    {
        foreach (DataGridViewCell cell in row.Cells)
        {
            // Check if the cell is not in the first column (ID column)
            if (cell.ColumnIndex != 0 && cell.Value != null &&
            cell.Value.ToString().ToLower().Contains(searchTerm.ToLower()))
            {
                row.Selected = true;
                return; // Exit the method if a match is found
            }
            else if (cell.ColumnIndex == 0 && cell.Value != null &&
            cell.Value.ToString().ToLower().Contains(searchTerm.ToLower()))
            {
                MessageBox.Show("Search in ID column is not allowed.", "Error", MessageBoxButtons.OK,
                MessageBoxIcon.Error);
                return; // Exit the method to prevent further search in ID column
            }
        }
    }
}

```

```

// Display a message if the search term is not found
    MessageBox.Show("No matching records found.", "Search Results", MessageBoxButtons.OK,
    MessageBoxIcon.Information);
}

private void label3_Click(object sender, EventArgs e)
{
}

private void s_order_dataGridView_SelectionChanged_2(object sender, EventArgs e)
{
    if (s_order_dataGridView.SelectedRows.Count > 0)
    {
        int rowIndex = s_order_dataGridView.SelectedRows[0].Index;
        text_s_order_id.Text = s_order_dataGridView.Rows[rowIndex].Cells[0].Value.ToString();
        text_supplier_id.Text = s_order_dataGridView.Rows[rowIndex].Cells[1].Value.ToString();
        dateTimePicker1.Value = Convert.ToDateTime(s_order_dataGridView.Rows[rowIndex].Cells[2].Value);
        text_total.Text = s_order_dataGridView.Rows[rowIndex].Cells[3].Value.ToString();
    }
}

private void btn_filter_Click(object sender, EventArgs e)
{
    DateTime fromDate = dateTimePickerFrom.Value.Date;
    DateTime toDate = dateTimePickerTo.Value.Date.AddDays(1).AddSeconds(-1); // End of the selected day

    // Validate the selected date range
    if (toDate < fromDate)
    {
        MessageBox.Show("End date must be after the start date.", "Validation Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        return;
    }

    // Filter the DataGridView based on the selected date range
    DataView dv = dt.DefaultView;
    string filterExpression = $"date >= #{fromDate.ToString("MM/dd/yyyy")}# AND date <= #{
    toDate.ToString("MM/dd/yyyy")}#";
    dv.RowFilter = filterExpression;
    s_order_dataGridView.DataSource = dv;
}

private void btn_ClearFilter_Click(object sender, EventArgs e)
{
    // Clear the filter and display the entire dataset in the DataGridView
    DataView dv = dt.DefaultView;
    dv.RowFilter = "";
    s_order_dataGridView.DataSource = dv;
}

private void total_Click(object sender, EventArgs e)
{
}

```

```

double total = 0;

// Loop through each row in the DataGridView
for (int i = 0; i < s_order_dataGridView.Rows.Count; i++)
{
    // Check if the cell value in the 'Total' column is not null or empty
    if (s_order_dataGridView.Rows[i].Cells[3].Value != null &&
s_order_dataGridView.Rows[i].Cells[3].Value.ToString() != "")
    {
        // Parse the cell value to a double and add it to the total
        total += double.Parse(s_order_dataGridView.Rows[i].Cells[3].Value.ToString());
    }
}

// Display the total in the TextBox
Cal_Total.Text = total.ToString();

// Set the TextBox text to bold and red color
Cal_Total.Font = new Font(Cal_Total.Font, FontStyle.Bold);
Cal_Total.ForeColor = Color.Blue;
}

private void label2_Click(object sender, EventArgs e)
{
}
}
}

```

Supplier Order Details Form

Supplier Order Details

Supplier Order Details

SEARCH :

Supplier Order Id :

Supplier Name :

Fruit Name :

Variety :

Quantity :

Unit :

Price :

	O_ID	S_Name	F_Name	Var
*				

Supplier Order Details

Supplier Order Details

SEARCH :

Supplier Order Id :

Supplier Name :

Fruit Name :

Variety :

Quantity :

Unit :

Price :

	O_ID	S_Name	F_Name	Var
*				

Error

Price cannot contains Letter.

OK

Supplier Order Details Form Coding

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Data.SqlClient;
using System.Diagnostics;
using System.Windows.Forms;
using static System.Windows.Forms.VisualStyles.VisualStyleElement;

namespace DNJ_Inventory_System
{
    public partial class supplier_order_details : Form
    {
        private const Keys k_copy = Keys.Control | Keys.C;
        private const Keys k_paste = Keys.Control | Keys.V;

        SqlConnection con = new SqlConnection(@"Data Source=LAPTOP-7O11E2P5\SQLEXPRESS;Initial
Catalog=dnjsystem;User ID=tanya;Password=1234;Encrypt=True;TrustServerCertificate=True");
        SqlDataAdapter adapter;
        DataTable dt;
        private string connectionString;
        private IEnumerable<string> allsupplierordernames;

        public supplier_order_details()
        {
            InitializeComponent();
        }

        protected override bool ProcessCmdKey(ref Message msg, Keys keyData)
        {
            if ((keyData == k_copy) || (keyData == k_paste))
            {
                return true;
            }
            else
            {
                return base.ProcessCmdKey(ref msg, keyData);
            }
        }

        private void Getsupplier_order_id()
        {
            string orderid;
            string query = "select o_id from s_order_details order by o_id Desc";
            con.Open();
            SqlCommand cmd = new SqlCommand(query, con);
            SqlDataReader dr = cmd.ExecuteReader();
            if (dr.Read())
            {
                int idr = int.Parse(dr[0].ToString()) + 1;
            }
        }
    }
}
```

```

orderid = idr.ToString("00000");
    }
    else if (Convert.IsDBNull(dr))
    {
        orderid = ("00001");
    }
    else
    {
        orderid = ("00001");
    }
    con.Close();
    text_order_id.Text = orderid.ToString();
}

private void supplier_order_details_Load(object sender, EventArgs e)
{
    Getsupplier_order_id();
    try
    {
        using (SqlConnection con = new SqlConnection(@"Data Source=LAPTOP-7O11E2P5\SQLEXPRESS;Initial
Catalog=dnjsystem;User ID=tanya;Password=1234;Encrypt=True;TrustServerCertificate=True"))
        {
            adapter = new SqlDataAdapter("SELECT * FROM s_order_details", con);
            dt = new DataTable();

            con.Open();

            // Move the data retrieval code here
            adapter.Fill(dt);
            s_order_details_dataGridView.DataSource = dt;

            // Also, fill the dataset here
            this.s_order_detailsTableAdapter.Fill(this.dnjsystemDataSet.s_payment_details);
            // Now, set the SelectionMode and assign the event handler
            s_order_details_dataGridView.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
            s_order_details_dataGridView.SelectionChanged += s_order_details_dataGridView_SelectionChanged;

        }

        // Now, the rest of your initialization code
        Getsupplier_order_id();

        text_fruit_name.Text = " ";
        text_variety.Text = " ";
        text_quantity.Text = " ";
        text_price.Text = " ";
    }
    catch (Exception)
    {
        // Handle exceptions here
    }
}

```

```

private void btn_update_Click(object sender, EventArgs e)
{
    string order_id = text_order_id.Text;

    bool order_idExists = CheckIfIdExists(order_id);

    if (order_idExists)
    {
        if (text_order_id.Text != "" && text_supplier_name.Text != "" && text_fruit_name.Text != "" && text_variety.Text
        != "" && text_quantity.Text != "" && text_unit.Text != "" && text_price.Text != "")
        {
            string query = "UPDATE s_order_details SET s_name=@s_name, f_name=@f_name, variety=@variety,
quant=@quant, unit=@unit, price=@price WHERE o_id=@o_id";
            SqlCommand cmd = new SqlCommand(query, con);
            cmd.Parameters.AddWithValue("@o_id", Convert.ToInt32(order_id));
            cmd.Parameters.AddWithValue("@s_name", text_supplier_name.Text);
            cmd.Parameters.AddWithValue("@f_name", text_fruit_name.Text);
            cmd.Parameters.AddWithValue("@variety", text_variety.Text);
            cmd.Parameters.AddWithValue("@quant", Convert.ToInt32(text_quantity.Text));
            cmd.Parameters.AddWithValue("@unit", (text_unit.Text));
            cmd.Parameters.AddWithValue("@price", Convert.ToInt32(text_price.Text));

            con.Open();
            cmd.ExecuteNonQuery();
            con.Close();
            display_data();
            ClearTextBoxes();

            MessageBox.Show("Data Successfully Updated", "Success", MessageBoxButtons.OK,
            MessageBoxIcon.Information);
            Getsupplier_order_id();
        }
        else
        {
            MessageBox.Show("Please select row which you want to update", "Empty", MessageBoxButtons.OK,
            MessageBoxIcon.Warning);
        }
    }
    else
    {
        if (text_order_id.Text == "" || text_supplier_name.Text == "" || text_fruit_name.Text == "" || text_variety.Text ==
        "" || text_quantity.Text == "" || text_unit.Text == "" || text_price.Text == "")
        {
            MessageBox.Show("Please fill in all the required fields", "Empty Fields", MessageBoxButtons.OK,
            MessageBoxIcon.Warning);
        }
        else
        {
            MessageBox.Show("Order ID, does not exist.", "Not Found", MessageBoxButtons.OK,
            MessageBoxIcon.Warning);
        }
    }
}

```

```

private bool CheckIfIdExists(string id)
{
    foreach (DataRow row in dt.Rows)
    {
        if (row[0].ToString() == id)
        {
            return true;
        }
    }
    return false;
}

private void ClearTextBoxes()
{
    text_order_id.Text = "";
    text_supplier_name.Text = "";
    text_fruit_name.Text = "";
    text_variety.Text = "";
    text_quantity.Text = "";
    text_unit.Text = "";
    text_price.Text = "";
}

private void display_data()
{
    string query = "SELECT * FROM s_order_details";
    con.Open();
    SqlDataAdapter da = new SqlDataAdapter(query, con);
    dt.Clear();
    da.Fill(dt);
    s_order_details_dataGridView.DataSource = dt;
    con.Close();
}

private void btn_save_Click(object sender, EventArgs e)
{
    string order_id = text_order_id.Text;

    bool order_idExists = CheckIfIdExists(order_id, "o_id");

    if (order_idExists)
    {
        MessageBox.Show("Order ID, already exists", "Error", MessageBoxButtons.OK, MessageBoxIcon.Warning);
    }
    else
    {
        if (text_order_id.Text != "" && text_supplier_name.Text != "" && text_fruit_name.Text != "" &&
text_variety.Text != "" && text_quantity.Text != "" && text_price.Text != "")
        {
            try
            {
                string query = "INSERT INTO s_order_details VALUES(@o_id, @s_name,@f_name, @variety, @quant,
@unit, @price)";

```

```

SqlCommand cmd1 = new SqlCommand(query, con);
    cmd1.Parameters.AddWithValue("@o_id", Convert.ToInt32(text_order_id.Text));
    cmd1.Parameters.AddWithValue("@s_name", text_supplier_name.Text);
    cmd1.Parameters.AddWithValue("@f_name", text_fruit_name.Text);
    cmd1.Parameters.AddWithValue("@variety", text_variety.Text);
    cmd1.Parameters.AddWithValue("@quant", Convert.ToInt32(text_quantity.Text));
    cmd1.Parameters.AddWithValue("@unit", text_unit.Text);
    cmd1.Parameters.AddWithValue("@price", Convert.ToInt32(text_price.Text));

    con.Open();
    cmd1.ExecuteNonQuery();
    MessageBox.Show("Data saved Successfully.", "Success", MessageBoxButtons.OK,
    MessageBoxIcon.Information);
    con.Close();
    display_data();
    ClearTextBoxes();
}
catch (Exception ex)
{
    MessageBox.Show("Error: " + ex.Message, "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
}
else
{
    MessageBox.Show("Please Fill Empty Fields", "Suggestion", MessageBoxButtons.OK,
    MessageBoxIcon.Warning);
}
}

private bool CheckIfIdExists(string id, string columnName)
{
    foreach (DataRow row in dt.Rows)
    {
        if (row[columnName].ToString() == id)
        {
            return true;
        }
    }
    return false;
}

private void text_fruit_name_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsLetter(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show("A Fruit Name cannot contains Number.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        text_fruit_name.Focus();
    }
}

```

```

private void text_order_id_TextChanged(object sender, EventArgs e)
{
    text_fruit_name.Text = "";
}

private void text_order_id_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsDigit(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show(" Order ID cannot contains Letter.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

private void text_supplier_id_KeyPress(object sender, KeyPressEventArgs e)
{
}

private void text_order_id_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_supplier_name.Focus();
    }
}

private void text_supplier_id_KeyDown(object sender, KeyEventArgs e)
{
}

private void text_fruit_name_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_variety.Focus();
    }
}

private void text_variety_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_quantity.Focus();
    }
}

private void text_quantity_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {

```

```

text_unit.Focus();
    }
}
private void text_price_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar) && !char.IsDigit(e.KeyChar) && e.KeyChar != '.')
    {
        e.Handled = true;
        MessageBox.Show("Price cannot contains Letter.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
        text_quantity.Focus();
    }

    // Allow only one decimal point
    if (e.KeyChar == '.' && (sender as System.Windows.Forms.TextBox).Text.Contains("."))
    {
        e.Handled = true;
        MessageBox.Show("price can only contain one decimal point.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        text_quantity.Focus();
    }
}
private void s_order_details_dataGridView_CellEnter(object sender, DataGridViewCellEventArgs e)
{
    if (s_order_details_dataGridView.SelectedRows.Count > 0)
    {
        int rowIndex = s_order_details_dataGridView.SelectedRows[0].Index;
        {
            text_order_id.Text = s_order_details_dataGridView.CurrentRow.Cells[0].Value.ToString();
            text_supplier_name.Text = s_order_details_dataGridView.CurrentRow.Cells[1].Value.ToString();
            text_fruit_name.Text = s_order_details_dataGridView.CurrentRow.Cells[2].Value.ToString();
            text_variety.Text = s_order_details_dataGridView.CurrentRow.Cells[3].Value.ToString();
            text_quantity.Text = s_order_details_dataGridView.CurrentRow.Cells[4].Value.ToString();
            text_unit.Text = s_order_details_dataGridView.CurrentRow.Cells[5].Value.ToString();
            text_price.Text = s_order_details_dataGridView.CurrentRow.Cells[6].Value.ToString();
        }
    }
}

private void btn_refresh_Click(object sender, EventArgs e)
{
    display_data();
    text_supplier_name.Text = "";
    text_fruit_name.Text = "";
    text_variety.Text = "";
    text_quantity.Text = "";
    text_unit.Text = "";
    text_price.Text = "";
    Getsupplier_order_id();

}
private void text_quantity_TextChanged(object sender, EventArgs e)
{
}

```

```

private void text_unit_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsLetter(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show("Unit cannot contains Number.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        text_unit.Focus();
    }
}

private void text_unit_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_price.Focus();
    }
}

private void Form1_Load(object sender, EventArgs e)
{
    // TODO: This line of code loads data into the 'dnjsystemDataSet.staff_details' table. You can move, or remove it,
    as needed.
    this.s_order_detailsTableAdapter.Fill(this.dnjsystemDataSet.s_payment_details);
}

private void s_order_details_dataGridView_SelectionChanged(object sender, EventArgs e)
{
    if (s_order_details_dataGridView.SelectedRows.Count > 0)
    {
        int rowIndex = s_order_details_dataGridView.SelectedRows[0].Index;
        text_order_id.Text = s_order_details_dataGridView.Rows[rowIndex].Cells[0].Value.ToString();
        text_supplier_name.Text = s_order_details_dataGridView.Rows[rowIndex].Cells[1].Value.ToString();
        text_fruit_name.Text = s_order_details_dataGridView.Rows[rowIndex].Cells[2].Value.ToString();
        text_variety.Text = s_order_details_dataGridView.Rows[rowIndex].Cells[3].Value.ToString();
        text_quantity.Text = s_order_details_dataGridView.Rows[rowIndex].Cells[4].Value.ToString();
        text_unit.Text = s_order_details_dataGridView.Rows[rowIndex].Cells[5].Value.ToString();
        text_price.Text = s_order_details_dataGridView.Rows[rowIndex].Cells[6].Value.ToString();
    }
}

private void text_variety_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsLetter(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show("Variety cannot contains Number.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        text_variety.Focus();
    }
}

```

```

private void text_quantity_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar) && !char.IsDigit(e.KeyChar) && e.KeyChar != '.')
    {
        e.Handled = true;
        MessageBox.Show("Quantity cannot contains Letter.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        text_quantity.Focus();
    }

    // Allow only one decimal point
    if (e.KeyChar == '.' && (sender as System.Windows.Forms.TextBox).Text.Contains("."))
    {
        e.Handled = true;
        MessageBox.Show("Quantity can only contain one decimal point.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        text_quantity.Focus();
    }
}

private void text_supplier_id_TextChanged(object sender, EventArgs e)
{
}

private void comboBox_supplier_name_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsLetter(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show("A suppliers Name cannot contains Number.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        text_supplier_name.Focus();
    }
}

private void text_supplier_name_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_fruit_name.Focus();
    }
}

private void text_supplier_name_SelectedIndexChanged(object sender, EventArgs e)
{
}

private void text_search_TextChanged(object sender, EventArgs e)
{
}

```

```

// Get the search keyword from the TextBox
string keyword = text_search.Text;

// Check if the keyword is not empty
if (!string.IsNullOrEmpty(keyword))
{
    // Filter the DataGridView based on the keyword
    DataView dv = dt.DefaultView;
    string filterExpression = $"s_name LIKE '%{keyword}%' OR f_name LIKE '%{keyword}%' OR variety LIKE '%{keyword}%' OR Convert(quant, 'System.String') LIKE '%{keyword}%' OR unit LIKE '%{keyword}%' OR Convert(price, 'System.String') LIKE '%{keyword}%'";
    Console.WriteLine("Filter Expression: " + filterExpression); // Debugging
    dv.RowFilter = filterExpression;
    Console.WriteLine("Filtered Rows Count: " + dv.Count); // Debugging
    s_order_details_dataGridView.DataSource = dv;
}
else
{
    // If the search TextBox is empty, show all data
    DataView dv = dt.DefaultView;
    dv.RowFilter = ""; // Clear any existing filter
    s_order_details_dataGridView.DataSource = dv;
}
}

private void Label2_Click(object sender, EventArgs e)
{
}

private void Text_supplier_name_TextChanged(object sender, EventArgs e)
{
}

private void quantity_Click(object sender, EventArgs e)
{
}

private void unit_Click(object sender, EventArgs e)
{
}

private void text_supplier_name_KeyDown_1(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_fruit_name.Focus();
    }
}
}
}

```

Supplier Payment Details Form

supplier_payment_details

Supplier Payment Details

SEARCH :

Payment Id :

Supplier Name :

Date :

Amount :

Transaction Mode :

Account Holder Name :

Payment Contact Number :

	p_id	s_name	date
*			

SAVE

UPDATE

REFRESH

PRINT PREVIEW

PRINT

supplier_payment_details

Supplier Payment Details

SEARCH :

Payment Id : 00001

Supplier Name : Tanya

Date : 20 April 2020

Amount : 500

Transaction Mode : online

Account Holder Name :

Payment Contact Number :

	p_id	s_name	date
*			

Missing Information

!

Please provide both account holder name and contact number for online transaction

OK

SAVE

UPDATE

REFRESH

PRINT

Supplier Payment Details Form Coding

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace DNJ_Inventory_System
{
    public partial class supplier_payment_details : Form
    {
        private const Keys k_copy = Keys.Control | Keys.C;
        private const Keys k_paste = Keys.Control | Keys.V;

        SqlConnection con = new SqlConnection(@"Data Source=LAPTOP-7O11E2P5\SQLEXPRESS;Initial
        Catalog=dnjssystem;User ID=tanya;Password=1234;Encrypt=True;TrustServerCertificate=True");
        SqlDataAdapter adapter;
        DataTable dt;
        private string connectionString;
        private IEnumerable<string> allsupplierordernames;

        public supplier_payment_details()
        {
            InitializeComponent();
        }
        protected override bool ProcessCmdKey(ref Message msg, Keys keyData)
        {
            if ((keyData == k_copy) || (keyData == k_paste))
            {
                return true;
            }
            else
            {
                return base.ProcessCmdKey(ref msg, keyData);
            }
        }
        private void Getsupplier_payment_id()
        {
            string orderid;
            string query = "select p_id from supplier_payment_details order by p_id Desc";
            con.Open();
            SqlCommand cmd = new SqlCommand(query, con);
            SqlDataReader dr = cmd.ExecuteReader();
            if (dr.Read())
            {
                int idr = int.Parse(dr[0].ToString()) + 1;
            }
        }
    }
}
```

```

orderid = idr.ToString("00000");
    }
    else if (Convert.IsDBNull(dr))
    {
        orderid = ("00001");

    }
    else
    {
        orderid = ("00001");
    }
    con.Close();
    text_payment_id.Text = orderid.ToString();
}
private void Form1_Load(object sender, EventArgs e)
{
    // TODO: This line of code loads data into the 'dnjsystemDataSet.supplier_payment_details' table. You can move, or
remove it, as needed.
    this.s_payment_detailsTableAdapter.Fill(this.dnjsystemDataSet.supplier_payment_details);
}
private void supplier_payment_details_Load(object sender, EventArgs e)
{
    Getsupplier_payment_id();
    try
    {
        using (SqlConnection con = new SqlConnection(@"Data Source=LAPTOP-7O11E2P5\SQLEXPRESS;Initial
Catalog=dnjsystem;User ID=tanya;Password=1234;Encrypt=True;TrustServerCertificate=True"))
        {
            adapter = new SqlDataAdapter("SELECT * FROM supplier_payment_details", con);
            dt = new DataTable();

            con.Open();

            // Move the data retrieval code here
            adapter.Fill(dt);
            s_payment_details_dataGridView.DataSource = dt;

            // Also, fill the dataset here
            this.s_payment_detailsTableAdapter.Fill(this.dnjsystemDataSet.supplier_payment_details);
            // Now, set the SelectionMode and assign the event handler
            s_payment_details_dataGridView.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
            s_payment_details_dataGridView.SelectionChanged += s_payment_details_dataGridView_SelectionChanged;

        }

        // Now, the rest of your initialization code
        Getsupplier_payment_id();

        text_supplier_name.Text = " ";
        dateTimePicker.Text = " ";
        text_amount.Text = " ";
        text_transaction_mode.Text = " ";
        text_account_holder_name.Text = " ";
        text_payment_number.Text = " ";

```

```

    }

    catch (Exception)
    {
        // Handle exceptions here
    }
}

private void s_payment_details_dataGridView_SelectionChanged(object sender, EventArgs e)
{
    throw new NotImplementedException();
}

private void btn_update_Click(object sender, EventArgs e)
{
    string p_id = text_payment_id.Text;

    bool p_idExists = CheckIfIdExists(p_id);

    if (p_idExists)
    {
        if (text_payment_id.Text != "" && text_supplier_name.Text != "" && dateTimePicker.Text != "" &&
text_amount.Text != "" && text_transaction_mode.Text != "" && text_account_holder_name.Text != "" &&
text_payment_number.Text != "")
        {
            string query = "UPDATE supplier_payment_details SET s_name=@s_name, date=@date,
amount=@amount, transaction_mode=@transaction_mode,
account_holder_name=@account_holder_name,contact=@contact WHERE p_id=@p_id";
            SqlCommand cmd = new SqlCommand(query, con);
            cmd.Parameters.AddWithValue("@p_id", Convert.ToInt32(p_id));
            cmd.Parameters.AddWithValue("@s_name", text_supplier_name.Text);
            cmd.Parameters.AddWithValue("@date", dateTimePicker.Text);
            cmd.Parameters.AddWithValue("@amount", (text_amount.Text));
            cmd.Parameters.AddWithValue("@transaction_mode", text_transaction_mode.Text);
            cmd.Parameters.AddWithValue("@account_holder_name", (text_account_holder_name.Text));
            cmd.Parameters.AddWithValue("@contact", (text_payment_number.Text));

            con.Open();
            cmd.ExecuteNonQuery();
            con.Close();
            display_data();
            ClearTextBoxes();

            MessageBox.Show("Data Successfully Updated", "Success", MessageBoxButtons.OK,
MessageBoxIcon.Information);
            Getsupplier_payment_id();
        }
        else
        {
            MessageBox.Show("Please select row which you want to update", "Empty",
MessageBoxButtons.OK, MessageBoxIcon.Warning);
        }
    }
}

```

```

else
{
    if (text_payment_id.Text == "" || text_supplier_name.Text == "" || dateTimePicker.Text == "" ||
text_transaction_mode.Text == "" || text_amount.Text == "" || text_account_holder_name.Text == "" ||
text_payment_number.Text == "")
    {
        MessageBox.Show("Please fill in all the required fields", "Empty Fields", MessageBoxButtons.OK,
MessageBoxIcon.Warning);
    }
    else
    {
        MessageBox.Show("Order ID, does not exist.", "Not Found", MessageBoxButtons.OK,
MessageBoxIcon.Warning);
    }
}
}
private bool CheckIfIdExists(string id)
{
    foreach (DataRow row in dt.Rows)
    {
        if (row[0].ToString() == id)
        {
            return true;
        }
    }
    return false;
}

private void ClearTextBoxes()
{
    text_payment_id.Text = "";
    text_supplier_name.Text = " ";
    dateTimePicker.Text = "";
    text_amount.Text = " ";
    text_transaction_mode.Text = " ";
    text_account_holder_name.Text = " ";
    text_payment_number.Text = " ";
}

private void display_data()
{
    string query = "SELECT * FROM supplier_payment_details";
    con.Open();
    SqlDataAdapter da = new SqlDataAdapter(query, con);
    dt.Clear();
    da.Fill(dt);
    s_payment_details_dataGridView.DataSource = dt;
    con.Close();
}

private void btn_save_Click(object sender, EventArgs e)
{
    string p_id = text_payment_id.Text;

    bool p_idExists = CheckIfIdExists(p_id, "p_id");

```

```

if (p_idExists)
{
    MessageBox.Show("Order ID already exists", "Error", MessageBoxButtons.OK, MessageBoxIcon.Warning);
}
else
{
    if (text_payment_id.Text != "" && text_supplier_name.Text != "" && dateTimePicker.Text != "" &&
text_amount.Text != "" && text_transaction_mode.Text != "")
    {
        try
        {
            string query = "INSERT INTO supplier_payment_details VALUES(@p_id, @s_name,@date, @amount,
@transaction_mode, @account_holder_name, @contact)";
            SqlCommand cmd1 = new SqlCommand(query, con);
            cmd1.Parameters.AddWithValue("@p_id", Convert.ToInt32(p_id));
            cmd1.Parameters.AddWithValue("@s_name", text_supplier_name.Text);
            cmd1.Parameters.AddWithValue("@date", dateTimePicker.Text);
            cmd1.Parameters.AddWithValue("@amount", Convert.ToInt32(text_amount.Text));
            cmd1.Parameters.AddWithValue("@transaction_mode", text_transaction_mode.Text);

            // Check if the transaction mode is "Online"
            if (text_transaction_mode.Text == "online")
            {
                // Check if both contact number and account holder name are provided
                if (string.IsNullOrEmpty(text_account_holder_name.Text) ||
string.IsNullOrEmpty(text_payment_number.Text))
                {
                    MessageBox.Show("Please provide both account holder name and contact number for online
transaction", "Missing Information", MessageBoxButtons.OK, MessageBoxIcon.Warning);
                    return; // Exit the method, preventing further execution
                }

                cmd1.Parameters.AddWithValue("@account_holder_name", text_account_holder_name.Text);
                cmd1.Parameters.AddWithValue("@contact", text_payment_number.Text);
            }
            else
            {
                // For Cash mode, set these parameters as null
                cmd1.Parameters.AddWithValue("@account_holder_name", DBNull.Value);
                cmd1.Parameters.AddWithValue("@contact", DBNull.Value);
            }
            con.Open();
            cmd1.ExecuteNonQuery();
            MessageBox.Show("Data saved Successfully.", "Success", MessageBoxButtons.OK,
MessageBoxIcon.Information);
            con.Close();
            display_data();
            ClearTextBoxes();
        }
        catch (Exception ex)
        {
            MessageBox.Show("Error: " + ex.Message, "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
}

```

```

    }
    }
    else
    {
        MessageBox.Show("Please Fill Empty Fields", "Suggestion", MessageBoxButtons.OK,
        MessageBoxIcon.Warning);
    }
}
private bool CheckIfIdExists(string id, string columnName)
{
    foreach (DataRow row in dt.Rows)
    {
        if (row[columnName].ToString() == id)
        {
            return true;
        }
    }
    return false;
}
private void text_payment_id_TextChanged(object sender, EventArgs e)
{
    text_supplier_name.Text = "";
}

private void text_payment_id_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsDigit(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show(" Order ID cannot contains Letter.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
    }
}

private void text_payment_id_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_supplier_name.Focus();
    }
}

private void s_payment_details_dataGridView_CellEnter(object sender, DataGridViewCellEventArgs e)
{
    if (s_payment_details_dataGridView.SelectedRows.Count > 0)
    {
        int rowIndex = s_payment_details_dataGridView.SelectedRows[0].Index;
        {
            text_payment_id.Text = s_payment_details_dataGridView.CurrentRow.Cells[0].Value.ToString();
            text_supplier_name.Text = s_payment_details_dataGridView.CurrentRow.Cells[1].Value.ToString();
            dateTimePicker.Text = s_payment_details_dataGridView.CurrentRow.Cells[2].Value.ToString();
        }
    }
}

```

```

text_amount.Text = s_payment_details_dataGridView.CurrentRow.Cells[3].Value.ToString();
    text_transaction_mode.Text = s_payment_details_dataGridView.CurrentRow.Cells[4].Value.ToString();
    text_account_holder_name.Text = s_payment_details_dataGridView.CurrentRow.Cells[5].Value.ToString();
    text_payment_number.Text = s_payment_details_dataGridView.CurrentRow.Cells[6].Value.ToString();
}
}
}

private void btn_refresh_Click(object sender, EventArgs e)
{
    display_data();
    text_supplier_name.Text = " ";
    dateTimePicker.Text = "";
    text_amount.Text = " ";
    text_transaction_mode.Text = " ";
    text_account_holder_name.Text = " ";
    text_payment_number.Text = " ";
    Getsupplier_payment_id();
}

private void s_payment_details_dataGridView_SelectionChanged_1(object sender, EventArgs e)
{
    if (s_payment_details_dataGridView.SelectedRows.Count > 0)
    {
        int rowIndex = s_payment_details_dataGridView.SelectedRows[0].Index;
        text_payment_id.Text = s_payment_details_dataGridView.Rows[rowIndex].Cells[0].Value.ToString();
        text_supplier_name.Text = s_payment_details_dataGridView.Rows[rowIndex].Cells[1].Value.ToString();
        dateTimePicker.Text = s_payment_details_dataGridView.Rows[rowIndex].Cells[2].Value.ToString();
        text_amount.Text = s_payment_details_dataGridView.Rows[rowIndex].Cells[3].Value.ToString();
        text_transaction_mode.Text = s_payment_details_dataGridView.Rows[rowIndex].Cells[4].Value.ToString();
        text_account_holder_name.Text = s_payment_details_dataGridView.Rows[rowIndex].Cells[5].Value.ToString();
        text_payment_number.Text = s_payment_details_dataGridView.Rows[rowIndex].Cells[6].Value.ToString();
    }
}

private void text_search_TextChanged(object sender, EventArgs e)
{
    /// Get the search keyword from the TextBox
    string keyword = text_search.Text;

    // Check if the keyword is not empty
    if (!string.IsNullOrEmpty(keyword))
    {
        // Filter the DataGridView based on the keyword
        DataView dv = dt.DefaultView;

        string filterExpression = $"s_name LIKE '%{keyword}%' OR Convert(date, 'System.String') LIKE '%{keyword}%' OR
        Convert(amount, 'System.String') LIKE '%{keyword}%' OR transaction_mode LIKE '%{keyword}%' OR
        account_holder_name LIKE '%{keyword}%' OR Convert(contact, 'System.String') LIKE '%{keyword}%'";
    }
}

```

```

Console.WriteLine("Filter Expression: " + filterExpression); // Debugging
    dv.RowFilter = filterExpression;
    Console.WriteLine("Filtered Rows Count: " + dv.Count); // Debugging
    s_payment_details_dataGridView.DataSource = dv;
}
else
{
    // If the search TextBox is empty, show all data
    DataView dv = dt.DefaultView;
    dv.RowFilter = ""; // Clear any existing filter
    s_payment_details_dataGridView.DataSource = dv;
}

}

private void button1_Click(object sender, EventArgs e)
{
    MprintPreviewDialog.Document=MprintDocument;
    MprintPreviewDialog.ShowDialog();
}

private void MprintDocument_PrintPage(object sender, System.Drawing.Printing.PrintPageEventArgs e)
{
    e.Graphics.DrawString("DNJ MERCHANT AND COMMISSION AGENT ", new Font("Times New Roman", 25, FontStyle.Bold), Brushes.DarkRed, new Point(40, 50));
    e.Graphics.DrawString("INVOICE ", new Font("Times New Roman", 50, FontStyle.Bold), Brushes.Black, new Point(250, 150));

    e.Graphics.DrawString("Supplier Name : " + text_supplier_name.Text, new Font("Arial", 16, FontStyle.Bold), Brushes.Black, new Point(25, 300));
    e.Graphics.DrawString("Date : " + DateTime.Now, new Font("Arial", 16, FontStyle.Bold), Brushes.Black, new Point(550, 300));

    String dashline = "_____";
    e.Graphics.DrawString(dashline, new Font("Arial", 11, FontStyle.Bold), Brushes.DarkRed, new Point(0, 350));

    e.Graphics.DrawString("Supplier Name : " + text_supplier_name.Text, new Font("Arial", 17, FontStyle.Bold), Brushes.Black, new Point(25, 400));
    e.Graphics.DrawString("Date : " + DateTime.Now, new Font("Arial", 17, FontStyle.Bold), Brushes.Black, new Point(25, 475));
    e.Graphics.DrawString("Amount : " + text_amount.Text, new Font("Arial", 17, FontStyle.Bold), Brushes.Black, new Point(25, 550));
    e.Graphics.DrawString("Transaction Mode : " + text_transaction_mode.Text, new Font("Arial", 17, FontStyle.Bold), Brushes.Black, new Point(25, 625));
    e.Graphics.DrawString("Account Holder Name: " + text_account_holder_name.Text, new Font("Arial", 17, FontStyle.Bold), Brushes.Black, new Point(25, 700));
    e.Graphics.DrawString("Payment Contact Number: " + text_payment_number.Text, new Font("Arial", 17, FontStyle.Bold), Brushes.Black, new Point(25, 775));
    String dashline1 = "_____";
    e.Graphics.DrawString(dashline1, new Font("Arial", 11, FontStyle.Bold), Brushes.DarkRed, new Point(0, 950));
}

```

```

private void button2_Click(object sender, EventArgs e)
{
    MprintDocument.Print();
}

private void text_payment_number_TextChanged(object sender, EventArgs e)
{
    if (text_payment_number.TextLength == 10)
    {
        text_payment_number.ForeColor = Color.Black;
    }
    else
    {
        text_payment_number.ForeColor = Color.Red;
    }
}

private void text_payment_number_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar) && !char.IsDigit(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show("A phone Number cannot contains Letter.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        text_payment_number.Focus();
    }
}

private void text_supplier_name_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        dateTimePicker.Focus();
    }
}

private void dateTimePicker_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_amount.Focus();
    }
}

private void text_amount_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_transaction_mode.Focus();
    }
}

```

```

private void text_transaction_mode_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_account_holder_name.Focus();
    }
}

private void text_account_holder_name_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_payment_number.Focus();
    }
}

private void text_supplier_name_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsLetter(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show("A Supplier Name cannot contains Number.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        text_supplier_name.Focus();
    }
}

private void text_amount_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsDigit(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show("Amount cannot contains letters.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
        text_amount.Focus();
    }
}

private void text_transaction_mode_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsLetter(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show("A Transaction Mode cannot contains Numbers.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        text_transaction_mode.Focus();
    }
}

```

```

    }
}

private void text_account_holder_name_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsLetter(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show("Account holder Name cannot contains Number.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        text_account_holder_name.Focus();
    }
}

private void quantity_Click(object sender, EventArgs e)
{
}

private void price_Click(object sender, EventArgs e)
{
}

private void order_id_Click(object sender, EventArgs e)
{
}

private void label1_Click(object sender, EventArgs e)
{
}
}
}

```

Customer Personal Details Form

customer_personal_details

Customer Personal Details

SEARCH :

Customer Id : 00001

Name :

State :

City/Village :

District :

Address :

Contact :

	C_ID	C_Name	State	City
*				

SAVE **UPDATE** **REFRESH**

customer_personal_details

Customer Personal Details

SEARCH :

Customer Id : 00001

Name : Tanya

State : Maharashtra

City/Village : xya

District : Raigad

Address :

Contact : 5615121523

	C_ID	C_Name	State	City
*				

SAVE **UPDATE** **REFRESH**

Error

⚠ Contact number must be 10 digits and start with 6, 7, 8, or 9

OK

Customer Personal Details Form Coding

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace DNJ_Inventory_System
{
    public partial class customer_personal_details : Form
    {
        public customer_personal_details()
        {
            InitializeComponent();
        }
        private const Keys k_copy = Keys.Control | Keys.C;
        private const Keys k_paste = Keys.Control | Keys.V;
        protected override bool ProcessCmdKey(ref Message msg, Keys keyData) // to disable copy paste
        {
            if ((keyData == k_copy) || (keyData == k_paste))
            {
                return true;
            }
            else
            {
                return base.ProcessCmdKey(ref msg, keyData);
            }
        }

        SqlConnection con = new SqlConnection(@"Data Source=LAPTOP-7O11E2P5\SQLEXPRESS;Initial
Catalog=dnjsystem;User ID=tanya;Password=1234;Encrypt=True;TrustServerCertificate=True");
        SqlDataAdapter adapter;
        DataTable dt;

        public void Getcustomerid()
        {
            string cusid;
            string query = "select c_id from c_personal_details order by c_id Desc";
            con.Open();
            SqlCommand cmd = new SqlCommand(query, con);
            SqlDataReader dr = cmd.ExecuteReader();
            if (dr.Read())
            {
                int idr = int.Parse(dr[0].ToString()) + 1;
            }
        }
    }
}
```

```

cusid = idr.ToString("00000");
    }
    else if (Convert.IsDBNull(dr))
    {
        cusid = ("00001");

    }
    else
    {
        cusid = ("00001");
    }
    con.Close();
    text_id.Text = cusid.ToString();
}

private void label1_Click(object sender, EventArgs e)
{

}

private void c_personal_detailsBindingNavigatorSaveItem_Click(object sender, EventArgs e)
{
    this.Validate();
    this.c_personal_detailsBindingSource.EndEdit();
    this.tableAdapterManager.UpdateAll(this.dnjsystemDataSet);

}

private void Form1_Load(object sender, EventArgs e)
{
    // TODO: This line of code loads data into the 'dnjsystemDataSet.c_personal_details' table. You can move, or
remove it, as needed.
    this.c_personal_detailsTableAdapter.Fill(this.dnjsystemDataSet.c_personal_details);

}
private void customer_personal_details_Load(object sender, EventArgs e)
{
    Getcustomerid();
    try
    {
        using (SqlConnection con = new SqlConnection(@"Data Source=LAPTOP-7O11E2P5\SQLEXPRESS;Initial
Catalog=dnjsystem;User ID=tanya;Password=1234;Encrypt=True;TrustServerCertificate=True"))
        {
            adapter = new SqlDataAdapter("SELECT * FROM c_personal_details", con);
            dt = new DataTable();

            con.Open();
//Move the data retrieval code here
            adapter.Fill(dt);
            c_personal_details_dataGridView.DataSource = dt;

            //Also, fill the dataset here
            this.c_personal_detailsTableAdapter.Fill(this.dnjsystemDataSet.c_personal_details);
        }
    }
}

```

```

//Now, the rest of your initialization code
    Getcustomerid();

}

catch (Exception)
{
    // MessageBox.Show($"Error: {ex.Message}");
}

}

private void btn_update_Click(object sender, EventArgs e)
{
    string id = text_id.Text;
    bool idExists = false;

    foreach (DataGridViewRow row in c_personal_details_dataGridView.Rows)
    {
        if (row.Cells[0].Value != null && row.Cells[0].Value.ToString() == id)
        {
            idExists = true;
            break;
        }
    }

    if (idExists)
    {
        if (text_id.Text != "" && text_name.Text != "" && text_state.Text != "" && text_city.Text != "" &&
text_district.Text != "" && text_contact.Text != "")
        {
            string query = "UPDATE c_personal_details SET c_name=@c_name, state=@state, city=@city,
district=@district, address=@address, contact=@contact WHERE c_id=@c_id";
            SqlCommand cmd = new SqlCommand(query, con);
            cmd.Parameters.AddWithValue("@c_id", Convert.ToInt32(text_id.Text));
            cmd.Parameters.AddWithValue("@c_name", text_name.Text);
            cmd.Parameters.AddWithValue("@state", text_state.Text);
            cmd.Parameters.AddWithValue("@city", text_city.Text);
            cmd.Parameters.AddWithValue("@district", text_district.Text);
            cmd.Parameters.AddWithValue("@contact", text_contact.Text);

            // Check if address is provided, if not, set it to NULL
            if (string.IsNullOrEmpty(text_address.Text))
            {
                cmd.Parameters.AddWithValue("@address", DBNull.Value);
            }
            else
            {
                cmd.Parameters.AddWithValue("@address", text_address.Text);
            }
        }
    }
}

```

```

con.Open();
    cmd.ExecuteNonQuery();
    con.Close();

    display_data();

    text_id.Text = "";
    text_name.Text = "";
    text_state.Text = "";
    text_city.Text = "";
    text_district.Text = "";
    text_address.Text = "";
    text_contact.Text = "";

    MessageBox.Show("Data Successfully Updated", "Success", MessageBoxButtons.OK,
    MessageBoxIcon.Information);
    Getcustomerid();
    }
    else
    {
        MessageBox.Show("Please fill in all required fields", "Incomplete Data", MessageBoxButtons.OK,
    MessageBoxIcon.Warning);
    }
    }
    else
    {
        MessageBox.Show("ID Does Not Exist", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

private void display_data()
{
    string query = "SELECT * FROM c_personal_details";
    con.Open();
    SqlDataAdapter da = new SqlDataAdapter(query, con);
    DataTable dt = new DataTable();
    da.Fill(dt);
    c_personal_details_dataGridView.DataSource = dt;
    con.Close();
}

private void btn_save_Click(object sender, EventArgs e)
{
    SqlCommand comcheckid = new SqlCommand("select c_id from c_personal_details where c_id=" + text_id.Text
+ "", con);
    SqlDataAdapter sd = new SqlDataAdapter(comcheckid);
    DataTable dt = new DataTable();
    sd.Fill(dt);

    if (dt.Rows.Count > 0)
    {
        MessageBox.Show("Id is already exist", "Error", MessageBoxButtons.OK, MessageBoxIcon.Warning);
    }
}

```

```

{
    if (text_id.Text != "" && text_name.Text != "" && text_state.Text != "" && text_city.Text != "" &&
text_district.Text != "" && text_contact.Text != "")
    {
        // Validate contact number length
        // Validate contact number
        if (text_contact.Text.Length != 10 || !text_contact.Text.StartsWith("6") &&
!text_contact.Text.StartsWith("7") && !text_contact.Text.StartsWith("8") && !text_contact.Text.StartsWith("9"))
        {
            MessageBox.Show("Contact number must be 10 digits and start with 6, 7, 8, or 9", "Error",
MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return; // Exit the method if contact number is not valid
        }

        // Clear error provider
        errorProvider1.Clear();

        // Prepare the SQL query with parameters
        string query = "INSERT INTO c_personal_details (c_id, c_name, state, city, district, address, contact)
VALUES (@c_id, @c_name, @state, @city, @district, @address, @contact)";
        SqlCommand cmd1 = new SqlCommand(query, con);
        cmd1.Parameters.AddWithValue("@c_id", text_id.Text);
        cmd1.Parameters.AddWithValue("@c_name", text_name.Text);
        cmd1.Parameters.AddWithValue("@state", text_state.Text);
        cmd1.Parameters.AddWithValue("@city", text_city.Text);

        // Check if district is provided, if not, set it to NULL
        if (string.IsNullOrEmpty(text_district.Text))
        {
            cmd1.Parameters.AddWithValue("@district", DBNull.Value);
        }
        else
        {
            cmd1.Parameters.AddWithValue("@district", text_district.Text);
        }

        // Check if address is provided, if not, set it to NULL
        if (string.IsNullOrEmpty(text_address.Text))
        {
            cmd1.Parameters.AddWithValue("@address", DBNull.Value);
        }
        else
        {
            cmd1.Parameters.AddWithValue("@address", text_address.Text);
        }

        cmd1.Parameters.AddWithValue("@contact", text_contact.Text);

        con.Open();
        cmd1.ExecuteNonQuery();
        con.Close();
    }
}

```

```

MessageBox.Show("Data saved Successfully.");
    display_data();

    text_name.Text = "";
    text_state.Text = "";
    text_city.Text = "";
    text_district.Text = "";
    text_address.Text = "";
    text_contact.Text = "";
}
else
{
    MessageBox.Show("Please Fill Empty Fields", "Suggestion", MessageBoxButtons.OK,
    MessageBoxIcon.Warning);
}
}
}

private void text_contact_TextChanged(object sender, EventArgs e)
{
    if (text_contact.TextLength == 10)
    {
        text_contact.ForeColor = Color.Black;
    }
    else
    {
        text_contact.ForeColor = Color.Red;
    }
}

private void text_contact_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar) && !char.IsDigit(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show("A phone Number cannot contains Letter.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        text_contact.Focus();
    }
}

private void text_name_KeyPress(object sender, KeyPressEventArgs e)
{
}

private void text_id_TextChanged(object sender, EventArgs e)
{
    text_name.Text = "";
}

```

```

private void text_cusid_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsDigit(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show("Customer ID cannot contain Letter.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
    }
}

private void text_id_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_name.Focus();
    }
}

private void text_name_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_state.Focus();
    }
}

private void text_state_KeyDown(object sender, KeyEventArgs e)
{
}

private void text_city_KeyDown(object sender, KeyEventArgs e)
{
}

private void text_district_KeyDown(object sender, KeyEventArgs e)
{
}

private void text_address_KeyDown(object sender, KeyEventArgs e)
{
}

private void text_name_KeyPress1(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)

```

```

{
    e.Handled = true;
    MessageBox.Show("A Name cannot contains Number.", "Error", MessageBoxButtons.OK,
    MessageBoxIcon.Error);
    text_name.Focus();
}
}

private void c_personal_details_dataGridView_CellEnter(object sender, DataGridViewCellEventArgs e)
{
    text_id.Text = c_personal_details_dataGridView.CurrentRow.Cells[0].Value.ToString();
    text_name.Text = c_personal_details_dataGridView.CurrentRow.Cells[1].Value.ToString();
    text_state.Text = c_personal_details_dataGridView.CurrentRow.Cells[2].Value.ToString();
    text_city.Text = c_personal_details_dataGridView.CurrentRow.Cells[3].Value.ToString();
    text_district.Text = c_personal_details_dataGridView.CurrentRow.Cells[4].Value.ToString();
    text_address.Text = c_personal_details_dataGridView.CurrentRow.Cells[5].Value.ToString();
    text_contact.Text = c_personal_details_dataGridView.CurrentRow.Cells[6].Value.ToString();
}

private void btn_refresh_Click(object sender, EventArgs e)
{
    display_data();
    text_name.Text = "";
    text_state.Text = "";
    text_city.Text = "";
    text_district.Text = "";
    text_address.Text = "";
    text_contact.Text = "";
    Getcustomerid();
}

private void text_contact_Leave(object sender, EventArgs e)
{
    Regex re = new Regex(@"^[6789]\d*$");

    if (re.IsMatch(text_contact.Text.Trim()) == false || text_contact.Text.Length != 10)
    {
        errorProvider2.SetError(text_contact, "Invalid Mobile Number");
        text_contact.Focus();
    }
    else
    {
        errorProvider2.SetError(text_contact, string.Empty);
    }
}

private void text_city_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsLetter(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))

```

```

{
    e.Handled = true;
    MessageBox.Show("A City/Village cannot contains Number.", "Error", MessageBoxButtons.OK,
    MessageBoxIcon.Error);
    text_city.Focus();

}
}

private void text_state_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsLetter(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show("A State cannot contains Number.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        text_state.Focus();

    }
}

private void text_district_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsLetter(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show("A District cannot contains Number.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        text_district.Focus();

    }
}

private void ClearTextBoxes()
{
    text_name.Text = "";
    text_state.Text = "";
    text_city.Text = "";
    text_district.Text = "";
    text_address.Text = "";
    text_contact.Text = "";
}

private void text_search_TextChanged(object sender, EventArgs e)
{
    // Get the search keyword from the TextBox
    string keyword = text_search.Text;

    // Check if the keyword is not empty
    if (!string.IsNullOrEmpty(keyword))
    {

```

```

{
    // Filter the DataGridView based on the keyword
    DataView dv = dt.DefaultView;
    string filterExpression = $"c_name LIKE '%{keyword}%' OR state LIKE '%{keyword}%' OR city LIKE '%{keyword}%' OR district LIKE '%{keyword}%' OR address LIKE '%{keyword}%' OR Convert(contact, 'System.String') LIKE '%{keyword}%'";
    Console.WriteLine("Filter Expression: " + filterExpression); // Debugging
    dv.RowFilter = filterExpression;
    Console.WriteLine("Filtered Rows Count: " + dv.Count); // Debugging
    c_personal_details_dataGridView.DataSource = dv;
}
else
{
    // If the search TextBox is empty, show all data
    DataView dv = dt.DefaultView;
    dv.RowFilter = ""; // Clear any existing filter
    c_personal_details_dataGridView.DataSource = dv;
}
// Clear the textboxes
ClearTextboxes();
}

private void label2_Click(object sender, EventArgs e)
{
}

private void c_personal_details_dataGridView_CellContentClick(object sender, DataGridViewCellEventArgs e)
{
}

private void address_Click(object sender, EventArgs e)
{
}

private void text_address_TextChanged(object sender, EventArgs e)
{
}

private void text_city_KeyDown1(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_district.Focus();
    }
}

private void text_district_KeyDown1(object sender, KeyEventArgs e)
{
}

```

```
if (e.KeyCode == Keys.Enter)
{
    text_address.Focus();
}

private void text_address_KeyDown1(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_contact.Focus();
    }
}

private void text_state_KeyDown1(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_city.Focus();
    }
}

private void district_Click(object sender, EventArgs e)
{
}
}
```

Customer Purchase Form

customer_purchase

Customer Purchase

FROM : 20 April 2024 TO : 01 April 2024 **SEARCH**

Purchase Id : 00001
Customer Id : 00001
Date : 20 April 2024
Total :

	C_O_ID	C_ID	Date
*			

SAVE **UPDATE** **REFRESH** **TOTAL** 0 **CLEAR**

customer_purchase

Customer Purchase

FROM : 20 April 2024 TO : 01 April 2024 **SEARCH**

Purchase Id : 00001
Customer Id : 00001
Date : 20 April 2024
Total : 5632

	C_O_ID	C_ID	Date
*			

SAVE **UPDATE** **REFRESH** **TOTAL** 0 **CLEAR**

Validation Error

End date must be after the start date.

OK

Customer Purchase Form Coding

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace DNJ_Inventory_System
{
    public partial class customer_order : Form
    {
        private const Keys k_copy = Keys.Control | Keys.C;
        private const Keys k_paste = Keys.Control | Keys.V;

        SqlConnection con = new SqlConnection(@"Data Source=LAPTOP-7O11E2P5\SQLEXPRESS;Initial
Catalog=dnjsystem;User ID=tanya;Password=1234;Encrypt=True;TrustServerCertificate=True");
        SqlDataAdapter adapter;
        DataTable dt;
        private object c_id;

        public customer_order()
        {
            InitializeComponent();
        }
        protected override bool ProcessCmdKey(ref Message msg, Keys keyData)
        {
            if ((keyData == k_copy) || (keyData == k_paste))
            {
                return true;
            }
            else
            {
                return base.ProcessCmdKey(ref msg, keyData);
            }
        }
        private void Getcustomer_order_id()
        {
            string orderid;
            string query = "select c_o_id from customer_order order by c_o_id Desc";
            con.Open();
            SqlCommand cmd = new SqlCommand(query, con);
            SqlDataReader dr = cmd.ExecuteReader();
            if (dr.Read())
            {
                int idr = int.Parse(dr[0].ToString()) + 1;
                orderid = idr.ToString("00000");
            }
        }
    }
}
```

```

else if (Convert.IsDBNull(dr))
{
   orderid = ("00001");

}
else
{
   orderid = ("00001");
}
con.Close();
text_order_id.Text = orderid.ToString();
}
public void Getcustomer_id()
{
    string supid;
    string query = "select c_id from customer_order order by c_id Desc";
    con.Open();
    SqlCommand cmd = new SqlCommand(query, con);
    SqlDataReader dr = cmd.ExecuteReader();
    if (dr.Read())
    {
        int idr = int.Parse(dr[0].ToString()) + 1;
        supid = idr.ToString("00000");
    }
    else if (Convert.IsDBNull(dr))
    {
        supid = ("00001");
    }
    else
    {
        supid = ("00001");
    }
    con.Close();
    text_customer_id.Text = supid.ToString();
}

private void panell_Paint(object sender, PaintEventArgs e)
{
}

private void customer_order_Load(object sender, EventArgs e)
{
    try
    {
        // Generate the customer order ID when the form loads
        Getcustomer_order_id();

        using (SqlConnection con = new SqlConnection(@"Data Source=LAPTOP-
7O11E2P5\SQLEXPRESS;Initial Catalog=dnjsystem;User
ID=tanya;Password=1234;Encrypt=True;TrustServerCertificate=True"))
        {

```

```

adapter = new SqlDataAdapter("SELECT * FROM customer_order", con);
    dt = new DataTable();

    con.Open();
    adapter.Fill(dt);
    con.Close();

    // Bind the DataGridView directly to the DataTable
    c_order_dataGridView.DataSource = dt;

    // Set the SelectionMode and assign the event handler
    c_order_dataGridView.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
    c_order_dataGridView.SelectionChanged += c_order_dataGridView_SelectionChanged;
}

// Generate the customer ID when the form loads
Getcustomer_id();

// Now, the rest of your initialization code
text_total.Text = " ";
}
catch (Exception)
{
    // Handle exceptions here
}
}

```

```

private void c_order_dataGridView_SelectionChanged(object sender, EventArgs e)
{
    //throw new NotImplementedException();
}

private void Form_Load(object sender, EventArgs e)
{
    // TODO: This line of code loads data into the 'dnjsystemDataSet.customer_order' table. You can
    move, or remove it, as needed.
    this.c_order_TableAdapter.Fill(this.dnjsystemDataSet.customer_order);
}

private void c_order_dataGridView_SelectionChanged_1(object sender, EventArgs e)
{
    if (c_order_dataGridView.SelectedRows.Count > 0)
    {
        int rowIndex = c_order_dataGridView.SelectedRows[0].Index;
        text_order_id.Text = c_order_dataGridView.Rows[rowIndex].Cells[0].Value.ToString();
        text_customer_id.Text = c_order_dataGridView.Rows[rowIndex].Cells[1].Value.ToString();
        dateTimePicker1.Value =
Convert.ToDateTime(c_order_dataGridView.Rows[rowIndex].Cells[2].Value);
        text_total.Text = c_order_dataGridView.Rows[rowIndex].Cells[3].Value.ToString();
    }
}

```

```

private void btn_update_Click(object sender, EventArgs e)
{
    string c_o_id = text_order_id.Text;
    string customer_id = text_customer_id.Text;

    bool c_o_idExists = CheckIfIdExists(c_o_id);
    bool customer_idExists = CheckIfIdExists(customer_id);

    if (c_o_idExists)
    {
        if (text_order_id.Text != "" && text_customer_id.Text != "" && text_total.Text != "" &&
datePicker1.Text != "")
        {
            string query = "UPDATE customer_order SET c_id=@c_id, total=@total, date=@date WHERE
c_o_id=@c_o_id";
            SqlCommand cmd = new SqlCommand(query, con);
            cmd.Parameters.AddWithValue("@c_o_id", Convert.ToInt32(c_o_id));
            cmd.Parameters.AddWithValue("@c_id", Convert.ToInt32(c_id));
            cmd.Parameters.AddWithValue("@total", Convert.ToInt32(text_total.Text));
            cmd.Parameters.AddWithValue("@date", datePicker1.Text);

            con.Open();
            cmd.ExecuteNonQuery();
            con.Close();
            display_data();
            ClearTextBoxes();

            MessageBox.Show("Data Successfully Updated", "Success", MessageBoxButtons.OK,
MessageBoxIcon.Information);
            Getcustomer_order_id();
            Getcustomer_id();
        }
        else
        {
            MessageBox.Show("Please select row which you want to update", "Empty",
MessageBoxButtons.OK, MessageBoxIcon.Warning);
        }
    }
    else
    {
        if (text_order_id.Text == "" || (text_customer_id.Text == "" || datePicker1.Text == "" ||
text_total.Text == ""))
        {
            MessageBox.Show("Please fill in all the required fields", "Empty Fields", MessageBoxButtons.OK,
MessageBoxIcon.Warning);
        }
        else
        {
            MessageBox.Show("Order ID, does not exist.", "Not Found", MessageBoxButtons.OK,
MessageBoxIcon.Warning);
        }
    }
}

```

```

    }
    }
}

private bool CheckIfIdExists(string id)
{
    foreach (DataRow row in dt.Rows)
    {
        if (row[0].ToString() == id)
        {
            return true;
        }
    }
    return false;
}

private bool ClearTextBoxes(string id)
{
    foreach (DataRow row in dt.Rows)
    {
        if (row[0].ToString() == id)
        {
            return true;
        }
    }
    return false;
}

private void ClearTextBoxes()
{
    text_customer_id.Text = "";
    text_order_id.Text = "";
    dateTimePicker1.Text = "";
    text_total.Text = "";
}

private void display_data()
{
    string query = "SELECT * FROM customer_order";
    con.Open();
    SqlDataAdapter da = new SqlDataAdapter(query, con);
    dt.Clear();
    da.Fill(dt);
    c_order_dataGridView.DataSource = dt;
    con.Close();
}

private void btn_save_Click(object sender, EventArgs e)
{
    string order_id = text_order_id.Text;

    bool order_idExists = CheckIfIdExists(order_id, "c_o_id");
    bool customer_idExists = CheckIfIdExists(text_customer_id.Text, "c_id");

```

```

if (order_idExists || customer_idExists)
{
    MessageBox.Show("Order ID, already exists", "Error", MessageBoxButtons.OK, MessageBoxIcon.Warning);
}
else
{
    if ((text_order_id.Text == "" || text_customer_id.Text == "" || dateTimePicker1.Text != "" && text_total.Text !=
""))
    {
        try
        {
            string query = "INSERT INTO customer_order VALUES(@c_o_id,@c_id, @date,@total)";
            SqlCommand cmd1 = new SqlCommand(query, con);
            cmd1.Parameters.AddWithValue("@c_o_id", Convert.ToInt32(text_order_id.Text));
            cmd1.Parameters.AddWithValue("@c_id", Convert.ToInt32(text_customer_id.Text));
            cmd1.Parameters.AddWithValue("@date", dateTimePicker1.Text);
            cmd1.Parameters.AddWithValue("@total", Convert.ToInt32(text_total.Text));

            con.Open();
            cmd1.ExecuteNonQuery();
            MessageBox.Show("Data saved Successfully.", "Success", MessageBoxButtons.OK,
MessageBoxIcon.Information);
            con.Close();
            display_data();
            ClearTextBoxes();
        }
        catch (Exception ex)
        {
            MessageBox.Show("Error: " + ex.Message, "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
    else
    {
        MessageBox.Show("Please Fill Empty Fields", "Suggestion", MessageBoxButtons.OK,
MessageBoxIcon.Warning);
    }
}

private bool CheckIfIdExists(string id, string columnName)
{
    foreach (DataRow row in dt.Rows)
    {
        if (row[columnName].ToString() == id)
        {
            return true;
        }
    }
    return false;
}

```

```

}

private void text_order_id_TextChanged(object sender, EventArgs e)
{
    dateTimePicker1.Text = "";
}

private void text_order_id_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        dateTimePicker1.Focus();
    }
}

private void text_order_id_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsDigit(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show(" Order ID cannot contains Letter.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
    }
}

private void dateTimePicker1_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsDigit(e.KeyChar)
        && !char.IsLetter(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show(" Please select date, typing anything is not allowed.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
    }
}

private void text_total_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsDigit(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show(" Total cannot contains Letter.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
    }
    // Allow only one decimal point
    if (e.KeyChar == '.' && (sender as TextBox).Text.Contains("."))
    {

```

```

e.Handled = true;
    MessageBox.Show("total can only contain one decimal point.", "Error", MessageBoxButtons.OK,
    MessageBoxIcon.Error);

    }
}

private void dateTimePicker1_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_total.Focus();
    }
}

private void c_order_dataGridView_CellEnter(object sender, DataGridViewCellEventArgs e)
{
    if (c_order_dataGridView.SelectedRows.Count > 0)
    {
        int rowIndex = c_order_dataGridView.SelectedRows[0].Index;
        {
            text_order_id.Text = c_order_dataGridView.CurrentRow.Cells[0].Value.ToString();
            text_customer_id.Text = c_order_dataGridView.CurrentRow.Cells[1].Value.ToString();
            dateTimePicker1.Text = c_order_dataGridView.CurrentRow.Cells[2].Value.ToString();
            text_total.Text = c_order_dataGridView.CurrentRow.Cells[3].Value.ToString();

        }
    }
}

private void btn_refresh_Click(object sender, EventArgs e)
{
    display_data();
    dateTimePicker1.Text = "";
    text_total.Text = "";

    Getcustomer_order_id();
    Getcustomer_id();
}

private void btn_filter_Click(object sender, EventArgs e)
{
    DateTime fromDate = dateTimePickerFrom.Value.Date;
    DateTime toDate = dateTimePickerTo.Value.Date.AddDays(1).AddSeconds(-1); // End of the selected day

    // Validate the selected date range
    if (toDate < fromDate)
    {
        MessageBox.Show("End date must be after the start date.", "Validation Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        return;
    }

    // Filter the DataGridView based on the selected date range
    DataView dv = dt.DefaultView;

```

```

string filterExpression = $"date >= #{fromDate.ToString("MM/dd/yyyy")}# AND date <= #{
toDate.ToString("MM/dd/yyyy")}#";
    dv.RowFilter = filterExpression;
    c_order_dataGridView.DataSource = dv;
}

private void btn_ClearFilter_Click(object sender, EventArgs e)
{
    // Clear the filter and display the entire dataset in the DataGridView
    DataView dv = dt.DefaultView;
    dv.RowFilter = "";
    c_order_dataGridView.DataSource = dv;
}

private void total_Click(object sender, EventArgs e)
{
    double total = 0;

    // Loop through each row in the DataGridView
    for (int i = 0; i < c_order_dataGridView.Rows.Count; i++)
    {
        // Check if the cell value in the 'Total' column is not null or empty
        if (c_order_dataGridView.Rows[i].Cells[3].Value != null &&
c_order_dataGridView.Rows[i].Cells[3].Value.ToString() != "")
        {
            // Parse the cell value to a double and add it to the total
            total += double.Parse(c_order_dataGridView.Rows[i].Cells[3].Value.ToString());
        }
    }

    // Display the total in the TextBox
    Cal_Total.Text = total.ToString();

    // Set the TextBox text to bold and red color
    Cal_Total.Font = new Font(Cal_Total.Font, FontStyle.Bold);
    Cal_Total.ForeColor = Color.Blue;
}
}
}

```

Customer Purchase Details Form

customer_purchase_details

Customer Purchase Details

SEARCH :

Purchase Id :

Customer Name :

Fruit Name :

Variety :

Quantity :

Unit :

Price :

	O_ID	C_Name	F_Name
*			

customer_purchase_details

Customer Purchase Details

SEARCH :

Purchase Id :

Customer Name :

Fruit Name :

Variety :

Quantity :

Unit :

Price :

	O_ID	C_Name	F_Name
*			

Error

Price cannot contains Letter.

OK

Customer Purchase Details Form Coding

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace DNJ_Inventory_System
{
    public partial class customer_order_details : Form
    {

        private const Keys k_copy = Keys.Control | Keys.C;
        private const Keys k_paste = Keys.Control | Keys.V;

        SqlConnection con = new SqlConnection(@"Data Source=LAPTOP-7O11E2P5\SQLEXPRESS;Initial
Catalog=dnjsystem;User ID=tanya;Password=1234;Encrypt=True;TrustServerCertificate=True");
        SqlDataAdapter adapter;
        DataTable dt;
        private string connectionString;
        private IEnumerable<string> allsupplierordernames;

        public customer_order_details()
        {
            InitializeComponent();
        }
        protected override bool ProcessCmdKey(ref Message msg, Keys keyData)
        {
            if ((keyData == k_copy) || (keyData == k_paste))
            {
                return true;
            }
            else
            {
                return base.ProcessCmdKey(ref msg, keyData);
            }
        }

        private void Getcustomer_order_id()
        {
            string orderid;
            string query = "select o_id from customer_order_details order by o_id Desc";
            con.Open();
            SqlCommand cmd = new SqlCommand(query, con);
            SqlDataReader dr = cmd.ExecuteReader();
```

```

{
    int idr = int.Parse(dr[0].ToString()) + 1;
   orderid = idr.ToString("00000");
}
else if (Convert.IsDBNull(dr))
{
    orderid = ("00001");
}
else
{
    orderid = ("00001");
}
con.Close();
text_order_id.Text = orderid.ToString();
}
private void Form1_Load(object sender, EventArgs e)
{
    // TODO: This line of code loads data into the 'dnjsystemDataSet.customer_order_details' table. You can move, or
remove it, as needed.
    this.c_order_detailsTableAdapter.Fill(this.dnjsystemDataSet.customer_order_details);
}
private void customer_order_details_Load(object sender, EventArgs e)
{
    try
    {
        // Generate the customer order ID when the form loads
        Getcustomer_order_id();

        using (SqlConnection con = new SqlConnection(@"Data Source=LAPTOP-7011E2P5\SQLEXPRESS;Initial
Catalog=dnjsystem;User ID=tanya;Password=1234;Encrypt=True;TrustServerCertificate=True"))
        {
            adapter = new SqlDataAdapter("SELECT * FROM customer_order_details", con);
            dt = new DataTable();

            con.Open();

            // Move the data retrieval code here
            adapter.Fill(dt);
            c_order_details_dataGridView.DataSource = dt;

            // Also, fill the dataset here
            this.c_order_detailsTableAdapter.Fill(this.dnjsystemDataSet.customer_order_details);
            // Now, set the SelectionMode and assign the event handler
            c_order_details_dataGridView.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
            c_order_details_dataGridView.SelectionChanged += c_order_details_dataGridView_SelectionChanged;

        }

        // Now, the rest of your initialization code
        Getcustomer_order_id();
    }
}

```

```

catch (Exception)
{
    // Handle exceptions here
}

private void c_order_details_dataGridView_SelectionChanged(object sender, EventArgs e)
{
    //throw new NotImplementedException();
}

private void btn_update_Click(object sender, EventArgs e)
{
    string order_id = text_order_id.Text;

    bool order_idExists = CheckIfIdExists(order_id);
    if (order_idExists)
    {
        if (text_order_id.Text != "" && text_customer_name.Text != "" && text_fruit_name.Text != "" &&
text_variety.Text != "" && text_quantity.Text != "" && text_unit.Text != "" && text_price.Text != "")
        {
            string query = "UPDATE customer_order_details SET c_name=@c_name, f_name=@f_name,
variety=@variety, quant=@quant, unit=@unit, price=@price WHERE o_id=@o_id";
SqlCommand cmd = new SqlCommand(query, con);
cmd.Parameters.AddWithValue("@o_id", Convert.ToInt32(order_id));
cmd.Parameters.AddWithValue("@c_name", text_customer_name.Text);
cmd.Parameters.AddWithValue("@f_name", text_fruit_name.Text);
cmd.Parameters.AddWithValue("@variety", text_variety.Text);
cmd.Parameters.AddWithValue("@quant", Convert.ToInt32(text_quantity.Text));
cmd.Parameters.AddWithValue("@unit", (text_unit.Text));
cmd.Parameters.AddWithValue("@price", Convert.ToInt32(text_price.Text));

            con.Open();
            cmd.ExecuteNonQuery();
            con.Close();
            display_data();
            ClearTextBoxes();

            MessageBox.Show("Data Successfully Updated", "Success", MessageBoxButtons.OK,
MessageBoxIcon.Information);
            Getcustomer_order_id();
        }
        else
        {
            MessageBox.Show("Please select row which you want to update", "Empty", MessageBoxButtons.OK,
MessageBoxIcon.Warning);
        }
    }
    else
    {
        if (text_order_id.Text == "" || text_customer_name.Text == "" || text_fruit_name.Text == "" || text_variety.Text
== "" || text_quantity.Text == "" || text_unit.Text == "" || text_price.Text == "")
        {

```

```

MessageBox.Show("Please fill in all the required fields", "Empty Fields", MessageBoxButtons.OK,
MessageBoxIcon.Warning);
    }
    else
    {
        MessageBox.Show("Order ID, does not exist.", "Not Found", MessageBoxButtons.OK,
MessageBoxIcon.Warning);
    }
}
}
private bool CheckIfIdExists(string id)
{
    foreach (DataRow row in dt.Rows)
    {
        if (row[0].ToString() == id)
        {
            return true;
        }
    }
    return false;
}

private void ClearTextBoxes()
{
    text_order_id.Text = "";
    text_customer_name.Text = "";
    text_fruit_name.Text = "";
    text_variety.Text = "";
    text_quantity.Text = "";
    text_unit.Text = "";
    text_price.Text = "";
}

private void display_data()
{
    string query = "SELECT * FROM customer_order_details";
    con.Open();
    SqlDataAdapter da = new SqlDataAdapter(query, con);
    dt.Clear();
    da.Fill(dt);
    c_order_details_dataGridView.DataSource = dt;
    con.Close();
}

private void btn_save_Click(object sender, EventArgs e)
{
    string order_id = text_order_id.Text;

    bool order_idExists = CheckIfIdExists(order_id, "o_id");

    if (order_idExists)
    {
        MessageBox.Show("Order ID, already exists", "Error", MessageBoxButtons.OK, MessageBoxIcon.Warning);
    }
}

```

```

else
{
    if (text_order_id.Text != "" && text_customer_name.Text != "" && text_fruit_name.Text != "" &&
text_variety.Text != "" && text_quantity.Text != "" && text_price.Text != "")
    {
        try
        {
            string query = "INSERT INTO customer_order_details VALUES(@o_id, @c_name,@f_name, @variety,
@quant, @unit, @price)";
            SqlCommand cmd1 = new SqlCommand(query, con);
            cmd1.Parameters.AddWithValue("@o_id", Convert.ToInt32(text_order_id.Text));
            cmd1.Parameters.AddWithValue("@c_name", text_customer_name.Text);
            cmd1.Parameters.AddWithValue("@f_name", text_fruit_name.Text);
            cmd1.Parameters.AddWithValue("@variety", text_variety.Text);
            cmd1.Parameters.AddWithValue("@quant", Convert.ToInt32(text_quantity.Text));
            cmd1.Parameters.AddWithValue("@unit", text_unit.Text);
            cmd1.Parameters.AddWithValue("@price", Convert.ToInt32(text_price.Text));

            con.Open();
            cmd1.ExecuteNonQuery();
            MessageBox.Show("Data saved Successfully.", "Success", MessageBoxButtons.OK,
MessageBoxIcon.Information);
            con.Close();
            display_data();
            ClearTextBoxes();
        }
        catch (Exception ex)
        {
            MessageBox.Show("Error: " + ex.Message, "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
    else
    {
        MessageBox.Show("Please Fill Empty Fields", "Suggestion", MessageBoxButtons.OK,
MessageBoxIcon.Warning);
    }
}

private bool CheckIfIdExists(string id, string columnName)
{
    foreach (DataRow row in dt.Rows)
    {
        if (row[columnName].ToString() == id)
        {
            return true;
        }
    }
    return false;
}

private void text_fruit_name_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)

```

```

    && !char.IsLetter(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show("A Fruit Name cannot contains Number.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        text_fruit_name.Focus();

    }
}

private void text_order_id_TextChanged(object sender, EventArgs e)
{
    text_fruit_name.Text = "";
}

private void text_order_id_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsDigit(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show(" Order ID cannot contains Letter.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
    }
}

private void text_order_id_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_customer_name.Focus();
    }
}

private void text_fruit_name_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_variety.Focus();
    }
}

private void text_variety_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_quantity.Focus();
    }
}

```

```

private void text_quantity_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_unit.Focus();
    }
}

private void text_price_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar) && !char.IsDigit(e.KeyChar) && e.KeyChar != '.')
    {
        e.Handled = true;
        MessageBox.Show("Price cannot contains Letter.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
        text_quantity.Focus();
    }

    // Allow only one decimal point
    if (e.KeyChar == '.' && (sender as System.Windows.Forms.TextBox).Text.Contains("."))
    {
        e.Handled = true;
        MessageBox.Show("price can only contain one decimal point.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        text_quantity.Focus();
    }
}

private void c_order_details_dataGridView_CellEnter(object sender, DataGridViewCellEventArgs e)
{
    if (c_order_details_dataGridView.SelectedRows.Count > 0)
    {
        int rowIndex = c_order_details_dataGridView.SelectedRows[0].Index;
        {
            text_order_id.Text = c_order_details_dataGridView.CurrentRow.Cells[0].Value.ToString();
            text_customer_name.Text = c_order_details_dataGridView.CurrentRow.Cells[1].Value.ToString();
            text_fruit_name.Text = c_order_details_dataGridView.CurrentRow.Cells[2].Value.ToString();
            text_variety.Text = c_order_details_dataGridView.CurrentRow.Cells[3].Value.ToString();
            text_quantity.Text = c_order_details_dataGridView.CurrentRow.Cells[4].Value.ToString();
            text_unit.Text = c_order_details_dataGridView.CurrentRow.Cells[5].Value.ToString();
            text_price.Text = c_order_details_dataGridView.CurrentRow.Cells[6].Value.ToString();
        }
    }
}

private void btn_refresh_Click(object sender, EventArgs e)
{
    display_data();
    text_customer_name.Text = "";
    text_fruit_name.Text = "";
    text_variety.Text = "";
    text_quantity.Text = "";
    text_unit.Text = "";
    text_price.Text = "";
    Getcustomer_order_id();
}

```

```

}

private void text_unit_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsLetter(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show("Unit cannot contains Number.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
        text_unit.Focus();
    }
}

private void text_unit_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_price.Focus();
    }
}

private void c_order_details_dataGridView_SelectionChanged_1(object sender, EventArgs e)
{
    if (c_order_details_dataGridView.SelectedRows.Count > 0)
    {
        int rowIndex = c_order_details_dataGridView.SelectedRows[0].Index;
        text_order_id.Text = c_order_details_dataGridView.Rows[rowIndex].Cells[0].Value.ToString();
        text_customer_name.Text = c_order_details_dataGridView.Rows[rowIndex].Cells[1].Value.ToString();
        text_fruit_name.Text = c_order_details_dataGridView.Rows[rowIndex].Cells[2].Value.ToString();
        text_variety.Text = c_order_details_dataGridView.Rows[rowIndex].Cells[3].Value.ToString();
        text_quantity.Text = c_order_details_dataGridView.Rows[rowIndex].Cells[4].Value.ToString();
        text_unit.Text = c_order_details_dataGridView.Rows[rowIndex].Cells[5].Value.ToString();
        text_price.Text = c_order_details_dataGridView.Rows[rowIndex].Cells[6].Value.ToString();
    }
}

private void text_variety_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsLetter(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show("Variety cannot contains Number.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
        text_variety.Focus();
    }
}

private void text_quantity_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar) && !char.IsDigit(e.KeyChar) && e.KeyChar != '.')
    {

```

```

e.Handled = true;
    MessageBox.Show("Quantity cannot contains Letter.", "Error", MessageBoxButtons.OK,
    MessageBoxIcon.Error);
    text_quantity.Focus();
}

// Allow only one decimal point
if (e.KeyChar == '.' && (sender as System.Windows.Forms.TextBox).Text.Contains("."))
{
    e.Handled = true;
    MessageBox.Show("Quantity can only contain one decimal point.", "Error", MessageBoxButtons.OK,
    MessageBoxIcon.Error);
    text_quantity.Focus();
}
}

private void text_customer_name_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsLetter(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show("A suppliers Name cannot contains Number.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        text_customer_name.Focus();
    }
}

private void text_customer_name_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_fruit_name.Focus();
    }
}

private void text_search_TextChanged(object sender, EventArgs e)
{
    // Get the search keyword from the TextBox
    string keyword = text_search.Text;

    // Check if the keyword is not empty
    if (!string.IsNullOrEmpty(keyword))
    {
        // Filter the DataGridView based on the keyword
        DataView dv = dt.DefaultView;
        string filterExpression = $"c_name LIKE '%{keyword}%' OR f_name LIKE '%{keyword}%' OR variety LIKE '%{keyword}%' OR Convert(quant, 'System.String') LIKE '%{keyword}%' OR unit LIKE '%{keyword}%' OR Convert(price, 'System.String') LIKE '%{keyword}%'";
        Console.WriteLine("Filter Expression: " + filterExpression); // Debugging
    }
}

```

```

dv.RowFilter = filterExpression;
    Console.WriteLine("Filtered Rows Count: " + dv.Count); // Debugging
    c_order_details_dataGridView.DataSource = dv;
}
else
{
    // If the search TextBox is empty, show all data
    DataView dv = dt.DefaultView;
    dv.RowFilter = ""; // Clear any existing filter
    c_order_details_dataGridView.DataSource = dv;
}

}

private void text_customer_name_TextChanged(object sender, EventArgs e)
{
}

private void c_order_details_dataGridView_CellContentClick(object sender, DataGridViewCellEventArgs e)
{
}

private void text_quantity_TextChanged(object sender, EventArgs e)
{
}
}
}
}

```

Product Details Form

product_details

Product Details

SEARCH :

Fruit Id :

Fruit Name :

Variety :

Quantity :

	F_ID	F_Name	Variety
*			

product_details

Product Details

SEARCH :

Fruit Id :

Fruit Name :

Variety :

Quantity :

	F_ID	F_Name	Variety
*			

Error

Quantity cannot contains Letter.

OK

Product Details Form Coding

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace DNJ_Inventory_System
{
    public partial class product_details : Form
    {
        private const Keys k_copy = Keys.Control | Keys.C;
        private const Keys k_paste = Keys.Control | Keys.V;

        SqlConnection con = new SqlConnection(@"Data Source=LAPTOP-7O11E2P5\SQLEXPRESS;Initial
Catalog=dnjsystem;User ID=tanya;Password=1234;Encrypt=True;TrustServerCertificate=True");
        SqlDataAdapter adapter;
        DataTable dt;
        private string connectionString;
        private IEnumerable<string> allsupplierordernames;
        private object text_remaining_quantity;
        public product_details()
        {
            InitializeComponent();
        }

        protected override bool ProcessCmdKey(ref Message msg, Keys keyData)
        {
            if ((keyData == k_copy) || (keyData == k_paste))
            {
                return true;
            }
            else
            {
                return base.ProcessCmdKey(ref msg, keyData);
            }
        }

        private void Getcustomer_order_id()
        {
            string orderid;
            string query = "select f_id from product_details order by f_id Desc";
            con.Open();
            SqlCommand cmd = new SqlCommand(query, con);
            SqlDataReader dr = cmd.ExecuteReader();
            if (dr.Read())
            {
                int idr = int.Parse(dr[0].ToString()) + 1;
                orderid = idr.ToString("00000");
            }
        }
    }
}
```

```

    }
    else if (Convert.IsDBNull(dr))
    {
        orderid = ("00001");
    }
    else
    {
        orderid = ("00001");
    }
    con.Close();
    text_fruit_id.Text = orderid.ToString();
}
private void Form1_Load(object sender, EventArgs e)
{
    // TODO: This line of code loads data into the 'dnjsystemDataSet.product_details' table. You can move, or
remove it, as needed.
    this.product_detailsTableAdapter.Fill(this.dnjsystemDataSet.product_details);
}
private void product_details_Load(object sender, EventArgs e)
{
    Getcustomer_order_id();
    try
    {
        using (SqlConnection con = new SqlConnection(@"Data Source=LAPTOP-7O11E2P5\SQLEXPRESS;Initial
Catalog=dnjsystem;User ID=tanya;Password=1234;Encrypt=True;TrustServerCertificate=True"))
        {
            adapter = new SqlDataAdapter("SELECT * FROM product_details", con);
            dt = new DataTable();

            con.Open();

            // Move the data retrieval code here
            adapter.Fill(dt);
            product_details_dataGridView.DataSource = dt;

            // Also, fill the dataset here
            this.product_detailsTableAdapter.Fill(this.dnjsystemDataSet.product_details);
            // Now, set the SelectionMode and assign the event handler
            product_details_dataGridView.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
            product_details_dataGridView.SelectionChanged += product_details_dataGridView_SelectionChanged;

        }
    }
    // Now, the rest of your initialization code
    Getcustomer_order_id();

    text_fruit_name.Text = " ";
    text_variety.Text = " ";
    text_quantity.Text = " ";

}

```

```

catch (Exception)
{
    // Handle exceptions here
}

private void product_details_dataGridView_SelectionChanged(object sender, EventArgs e)
{
    if (product_details_dataGridView.SelectedRows.Count > 0)
    {
        int rowIndex = product_details_dataGridView.SelectedRows[0].Index;
        text_fruit_id.Text = product_details_dataGridView.Rows[rowIndex].Cells[0].Value.ToString();
        text_fruit_name.Text = product_details_dataGridView.Rows[rowIndex].Cells[1].Value.ToString();
        text_variety.Text = product_details_dataGridView.Rows[rowIndex].Cells[2].Value.ToString();
    }
}

private void btn_update_Click(object sender, EventArgs e)
{
    string fruit_id = text_fruit_id.Text;

    bool fruit_idExists = CheckIfIdExists(fruit_id);
    if (fruit_idExists)
    {
        if (text_fruit_id.Text != "" && text_fruit_name.Text != "" && text_variety.Text != "" && text_quantity.Text
!= "")
        {
            string query = "UPDATE product_details SET f_name=@f_name, variety=@variety, quant=@quant
WHERE f_id=@f_id";
            SqlCommand cmd = new SqlCommand(query, con);
            cmd.Parameters.AddWithValue("@f_id", Convert.ToInt32(fruit_id));

            cmd.Parameters.AddWithValue("@f_name", text_fruit_name.Text);
            cmd.Parameters.AddWithValue("@variety", text_variety.Text);
            cmd.Parameters.AddWithValue("@quant", Convert.ToInt32(text_quantity.Text));

            con.Open();
            cmd.ExecuteNonQuery();
            con.Close();
            display_data();
            ClearTextBoxes();

            MessageBox.Show("Data Successfully Updated", "Success", MessageBoxButtons.OK,
MessageBoxIcon.Information);
            Getcustomer_order_id();
        }
        else
        {
            MessageBox.Show("Please select row which you want to update", "Empty", MessageBoxButtons.OK,
MessageBoxIcon.Warning);
        }
    }
    else

```

```

{
    MessageBox.Show("Order ID, does not exist.", "Not Found", MessageBoxButtons.OK,
    MessageBoxIcon.Warning);
}
}

private bool CheckIfIdExists(string id)
{
    foreach (DataRow row in dt.Rows)
    {
        if (row[0].ToString() == id)
        {
            return true;
        }
    }
    return false;
}

private void ClearTextBoxes()
{
    text_fruit_id.Text = "";
    text_fruit_name.Text = "";
    text_variety.Text = "";
    text_quantity.Text = "";
}

private void display_data()
{
    string query = "SELECT * FROM product_details";
    con.Open();
    SqlDataAdapter da = new SqlDataAdapter(query, con);
    dt.Clear();
    da.Fill(dt);
    product_details_dataGridView.DataSource = dt;
    con.Close();
}

private void btn_save_Click(object sender, EventArgs e)
{
    string order_id = text_fruit_id.Text;

    bool order_idExists = CheckIfIdExists(order_id, "f_id");

    if (order_idExists)
    {
        MessageBox.Show("Order ID, already exists", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Warning);
    }
    else
    {
        if (text_fruit_id.Text != "" && text_fruit_name.Text != "" && text_variety.Text != "" && text_quantity.Text
        != "")

```

```

{
    try
    {
        string query = "INSERT INTO product_details VALUES(@f_id,@f_name, @variety, @quant)";
        SqlCommand cmd1 = new SqlCommand(query, con);
        cmd1.Parameters.AddWithValue("@f_id", Convert.ToInt32(text_fruit_id.Text));

        cmd1.Parameters.AddWithValue("@f_name", text_fruit_name.Text);
        cmd1.Parameters.AddWithValue("@variety", text_variety.Text);
        cmd1.Parameters.AddWithValue("@quant", Convert.ToInt32(text_quantity.Text));

        con.Open();
        cmd1.ExecuteNonQuery();
        MessageBox.Show("Data saved Successfully.", "Success", MessageBoxButtons.OK,
        MessageBoxIcon.Information);
        con.Close();
        display_data();
        ClearTextBoxes();
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error: " + ex.Message, "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
    }
    else
    {
        MessageBox.Show("Please Fill Empty Fields", "Suggestion", MessageBoxButtons.OK,
        MessageBoxIcon.Warning);
    }
}

private bool CheckIfIdExists(string id, string columnName)
{
    foreach (DataRow row in dt.Rows)
    {
        if (row[columnName].ToString() == id)
        {
            return true;
        }
    }
    return false;
}

private void product_details_dataGridView_CellEnter(object sender, DataGridViewCellEventArgs e)
{
    if (product_details_dataGridView.SelectedRows.Count > 0)
    {
        int rowIndex = product_details_dataGridView.SelectedRows[0].Index;
    }
}

```

```

text_fruit_id.Text = product_details_dataGridView.CurrentRow.Cells[0].Value.ToString();
    text_fruit_name.Text = product_details_dataGridView.CurrentRow.Cells[1].Value.ToString();
    text_variety.Text = product_details_dataGridView.CurrentRow.Cells[2].Value.ToString();
    text_quantity.Text = product_details_dataGridView.CurrentRow.Cells[3].Value.ToString();

    }
}

private void btn_refresh_Click(object sender, EventArgs e)
{
    display_data();

    text_fruit_name.Text = "";
    text_variety.Text = "";
    text_quantity.Text = "";

    Getcustomer_order_id();
}

private void text_search_TextChanged(object sender, EventArgs e)
{
    // Get the search keyword from the TextBox
    string keyword = text_search.Text;

    // Check if the keyword is not empty
    if (!string.IsNullOrEmpty(keyword))
    {
        // Filter the DataGridView based on the keyword
        DataView dv = dt.DefaultView;
        string filterExpression = $"f_name LIKE '%{keyword}%' OR variety LIKE '%{keyword}%' OR
Convert(quant, 'System.String') LIKE '%{keyword}%'";
        Console.WriteLine("Filter Expression: " + filterExpression); // Debugging
        dv.RowFilter = filterExpression;
        Console.WriteLine("Filtered Rows Count: " + dv.Count); // Debugging
        product_details_dataGridView.DataSource = dv;
    }
    else
    {
        // If the search TextBox is empty, show all data
        DataView dv = dt.DefaultView;
        dv.RowFilter = ""; // Clear any existing filter
        product_details_dataGridView.DataSource = dv;
    }
}

private void text_fruit_id_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_fruit_name.Focus();
    }
}

```

```

private void text_fruit_name_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_variety.Focus();
    }
}

private void text_quantity_KeyDown(object sender, KeyEventArgs e)
{
}

private void text_variety_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        text_quantity.Focus();
    }
}

private void text_fruit_id_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar) && !char.IsDigit(e.KeyChar) && e.KeyChar != '.')
    {
        e.Handled = true;
        MessageBox.Show("Fruit ID cannot contains Letter.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        text_fruit_name.Focus();
    }
}

private void text_fruit_name_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar)
        && !char.IsLetter(e.KeyChar)
        && !char.IsWhiteSpace(e.KeyChar))
    {
        e.Handled = true;
        MessageBox.Show("A Fruit Name cannot contains Number.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        text_variety.Focus();
    }
}

private void text_quantity_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar) && !char.IsDigit(e.KeyChar) && e.KeyChar != '.')
    {
        e.Handled = true;
        MessageBox.Show("Quantity cannot contains Letter.", "Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        text_quantity.Focus();
    }
}

```

```
// Allow only one decimal point
    if (e.KeyChar == '.' && (sender as
System.Windows.Forms.TextBox).Text.Contains("."))
    {
        e.Handled = true;
        MessageBox.Show("Quantity can only contain one decimal point.", "Error",
MessageBoxButtons.OK, MessageBoxIcon.Error);
        text_quantity.Focus();
    }
}
}
```

SYSTEM REQUIREMENTS

SOFTWARE SPECIFICATION:

OPERATING SYSTEM: WINDOWS 11

ENVIROMENT: VISUAL STUDIO 2022

BACKEND: MICROSOFT SQL SERVER MANAGEMENT STUDIO 2018

LANGUAGE: C#.NET

HARDWARE SPECIFICATION:

SYSTEM: ACER LAPTOP

RAM: 8GB

HARD DISK: 476.92GB

CONCLUSION

- The main advantages of our project are that it is very user friendly.
- Modification and searching for records are very easy.
- This is providing easy way to operator interact with database and manipulate data in database
- Good security.
- Manage all activity very systematic way

SCOPE FOR FUTURE ENHANCEMENT

The Fruit Inventory Management System project scope can be expanded to include several future enhancements to improve its functionality and effectiveness. Some possible enhancements include:

Integration with other system: The Fruit Inventory Management System can be integrated with other project management system, such as scheduling software, document management system, and accounting software, to provide a more comprehensive solution for Fruit Inventory Management.

3D Visualization: The fruit Inventory Management can be enhanced with 3D visualization tools to allow user to visualize the Fruit Inventory process in real-time. This would enable admin to identify potential issues and risks before they occur and make more informed decisions.

Automated reporting: The Fruit Inventory Management System can be configured to automatically generate reports on project progress, cost and performance. This would reduce the time and effort required to prepare these reports manually and provide more accurate and up-to-date information.

In conclusion, The Fruit Inventory Management System project scope can be expand to include several future enhancements that can significantly improve its functionality and effectiveness.

These enhancement can improve, collaboration, efficiency, transparency, and accountability, and help to deliver successful outcomes for all stakeholders.

BIBLIOGRAPHY

<https://www.youtube.com>

<https://www.google.co.in/>